Building Extensible Program Logics through Effect Handlers (Extended Version)

ZICHEN ZHANG, New York University, USA
SIMON ODDERSHEDE GREGERSEN, CISPA Helmholtz Center for Information Security, Germany
JOSEPH TASSAROTTI*, New York University, USA

One approach to reasoning about effectful programs is to use program logics that provide proof rules for various effects. However, *developing* program logics requires skills that are distinct from those needed for *using* program logics, making the development of new logics less accessible. Moreover, when developing new logics, it can be difficult to reuse components from prior logics or combine support for different effects.

In this paper, we propose an approach for building extensible program logics based on effect handlers. Our starting point is an expressive program logic for reasoning about programs written in a pure, sequential language with support for effect handlers. Within this language, we implement handlers that model concurrency, distributed execution, and crash-recovery behavior. Then, by proving properties of these handlers within the logic, we derive expressive rules for reasoning about these effects. In some cases, this approach leads to stronger reasoning rules than those found in prior program logics.

In addition, we develop a relational logic for proving contextual refinements between programs using effects. As with unary reasoning, handlers enable this relational logic to be developed in an extensible way.

CCS Concepts: • **Theory of computation** → **Separation logic**; **Logic and verification**; **Program verification**; *Control primitives*; *Concurrency*; *Distributed computing models*; • **Software and its engineering** → Software fault tolerance; Formal software verification.

Additional Key Words and Phrases: Iris, Contextual refinement, Prophecy variables, Crash recovery, Perennial

1 Introduction

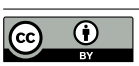
Program logics have proven to be powerful tools for program verification. As a result, a variety of program logics have been developed for a wide range of program features, including pointers [49], concurrency [31, 44, 45], weak memory [17, 33, 35, 40, 59, 60], crash recovery [13–15, 43, 48], distributed execution [34, 50, 65], and randomness [2, 3, 5–8, 26, 55], among others.

Traditionally, a program logic is developed by proving that a collection of reasoning rules is sound with respect to an operational or denotational semantics for a language. However, following this traditional approach is challenging for several reasons. First, the development of program logics requires skills that are distinct from those needed for *using* program logics, making the development of new logics less accessible. Second, when developing new logics, it can be difficult to reuse components or port a particular feature from a different logic. In reality, many important computer systems *combine* many of the features described in the previous paragraph, yet developing logics that provide support for such combinations of features requires significant work.

Recently, Vistrup et al. [64] proposed an alternative approach to developing program logics that addresses the reusability problem. Starting from a minimal, pure lambda calculus, they incrementally add effects to this language by giving a denotational semantics in terms of ITrees [67]. On the logic side, one gives rules that logically "interpret" or "handle" the events in the generated ITree.

*Also affiliated with Amazon Web Services. This paper does not reflect the views of Amazon Web Services.

Authors' Contact Information: Zichen Zhang, New York University, New York, USA, zichenzhang@nyu.edu; Simon Oddershede Gregersen, CISPA Helmholtz Center for Information Security, Saarbrücken, Germany, gregersen@cispa.de; Joseph Tassarotti, New York University, New York, USA, jt4767@nyu.edu.



This work is licensed under a Creative Commons Attribution 4.0 International License.

The soundness of the logic is established in a modular way by relating these logical handlers for events in the ITree to an interpretation function that "executes" the events. While this approach addresses the problem of reusability, it does not address the first issue on accessibility: it requires understanding the formalism of ITrees and their denotation, and the adequacy proofs require a form of reasoning that is different from the task of *using* the program logic to reason about programs.

This paper advocates for a third approach to developing program logics by using *effect handlers* [46, 47] to model all program effects. Effect handlers are a language feature that enables programmers to define custom effects in a modular and compositional way. Recent work has shown how to develop program logics for reasoning about effect handlers [18–21]. For example, using these logics, it is possible to verify an effect handler that implements a mutable state effect and derive a specification that resembles the usual separation logic rules for reasoning about pointers. As a result, reasoning about a client program that uses this state effect handler is just like carrying out a standard separation logic proof about a program that uses built-in primitive state. Because effect handlers are just regular programs, this approach results in a semantics that is executable by construction. However, prior program logics for effect handlers have considered a setting where effect handlers are added on top of a language that already has various forms of primitive effects built in, such as mutable references and concurrency. This makes sense for verifying examples that involve a subtle interaction between primitive effects and effect handlers, but the soundness proofs of these logics then combine the complexity of primitive effects and effect handlers.

In this work, we instead use effect handlers to bootstrap an expressive program logic for a range of effects. Our starting point is a minimal stateless core effect handler language called FicusLang. For reasoning about programs written in FicusLang, we develop Ficus, a separation logic for effect handlers that is an adaptation of an earlier logic called Hazel [19]. Using FicusLang, we then implement handlers to model mutable state, shared-memory concurrency, crash-recovery with durable state, and distributed execution over unreliable networks. For each effect, we apply Ficus to verify the handler implementations and obtain proof rules that are analogous to the reasoning principles derived in prior specialized program logics for these effects.

The handler-based approach even allows us to derive *stronger* proof rules than prior work. There are two main reasons. First, with the handler-based approach, we can build up effects in a hierarchical way, using earlier effects in the definition of handlers for later effects. Then, when verifying those later effects, we can use the proof rules from the earlier effects. For example, we show in §5 how to derive local *prophecy variables* [1, 32] from a simpler global prophecy by implementing local prophecy variables as an effect handler. Later, when implementing the handlers for crashes and recovery in §6, we attach prophecy variables to non-deterministic choices made during crashes, which allows client proofs to reason about when crashes will occur.

A second source of stronger proof rules arises from using effect handlers to implement non-standard versions of effects that are easier to reason about. For example, our handler for concurrency generates fewer interleavings than a standard operational semantics for concurrency. As a result, the "invariant-opening" rule that we obtain for this handler is stronger than the standard rule from most concurrent separation logics (CSLs). To justify the use of these non-standard semantics, we prove that they are equivalent in an appropriate sense to the standard semantics. To do so, we develop a new relational logic for effect handlers called RelFicus.

Contributions. To summarize, our work makes the following contributions:

- We extend Hazel [19] to develop Ficus, an extensible unary program logic based on effect handlers (§2). Ficus uses *protocols* to decompose reasoning about handlers and client code using handlers, and adds support for an extensible notion of *worlds* to share resources between client code (§3).

$$
\begin{aligned}
v &::= () \mid \mathbf{rec}\ f\ x.\ e \mid \cdots \mid \mathbf{cont}\ N \\
e &::= v \mid x \mid e\ e \mid \cdots \mid \mathbf{do}\ e \mid \S(N)[v] \mid (\mathbf{try}\ e\ \mathbf{with}\ v\ k \Rightarrow\ e \mid \mathbf{ret}\ v \Rightarrow e) \mid \mathbf{pick} \mid \mathbf{observe}\ e \\
K &::= \bullet \mid e\ K \mid K\ v \mid \cdots \mid \mathbf{do}\ K \mid (\mathbf{try}\ K\ \mathbf{with}\ v\ k \Rightarrow\ e \mid \mathbf{ret}\ v \Rightarrow e) \\
N &::= \bullet \mid e\ N \mid N\ v \mid \cdots \mid \mathbf{do}\ N
\end{aligned}
$$

Fig. 1. Syntax of values v, expressions e, evaluation contexts K, and neutral evaluation contexts N.

- We develop a relational logic, RelFicus, for proving contextual refinement in the presence of effect handlers (§4). RelFicus adapts Ficus protocols to the relational setting, similarly allowing for reuse and extensibility.
- As case studies, we show how to reconstruct and extend features from existing program logics using our effect handler approach. This includes: (1) A derivation of *local* prophecy variables out of global prophecies, along with an approach to *implicitly* make prophecies without annotating a program with prophecy operations (§5); (2) A logic for crash-recovery reasoning with asynchronous durable storage that recovers the features of Perennial [13], but with a simpler model, and novel support for *crash-aware* prophecy variables (§6); (3) A logic for distributed systems with IronFleet-style [27] atomic blocks (§7).

Our work is mechanized in the Iris separation logic framework [31] and the Rocq Prover.

2 Program Logics by Effect Handlers

This section provides an overview of how to build up a program logic using effect handlers. After introducing FicusLang, we describe the core features of Ficus, and use them to develop proof rules for reasoning about mutable state. When writing inference rules, we use two different styles. When the line dividing the premises from the conclusion is a standard horizontal bar, then this rule should be read as an implication in the meta-logic. When the horizontal bar is replaced by a $-\!\!*$, we use a rule of the form below as a notation for the separation logic entailment $P_1 * \cdots * P_n \vdash Q$.

$$\frac{P_1 \quad \cdots \quad P_n}{Q}\,{*}$$

2.1 The FicusLang Calculus

FicusLang is an ML-style lambda calculus with effect handlers. The syntax of FicusLang is shown in Figure 1. It uses call-by-value and right-to-left evaluation order. The expression $\mathbf{do}\ v$ raises an effect with value v, which will later be handled by the closest enclosing effect handler. Expression $\mathbf{try}\ e_0\ \mathbf{with}\ v_1\ k \Rightarrow\ e_1 \mid \mathbf{ret}\ v_2 \Rightarrow e_2$ installs a *shallow* effect handler for e_0: it evaluates e_0 until either e_0 raises an effect or becomes a value. For the first case, the handler will have access to the value v_1 raised by the effect and a continuation k that allows the handler to resume e_0 from the point the effect was raised. For the second case, the handler will obtain the result of e_0, a value v_2. This type of handler is called a shallow handler because it will disappear in both cases, and it must be reinstalled if the expression e_0 may raise effects multiple times.[1] Continuations are used by applying them like functions. To find the closest handler and capture the continuation when raising an effect, $\mathbf{do}\ v$ evaluates to an *active effect* $\S(\bullet)[v]$, which then gradually captures its surrounding neutral evaluation context using rule $N_1[\S(N_2)[v]] \rightarrow^* \S(N_1 +\!\!+ N_2)[v]$, until the immediate surrounding context becomes a $\mathbf{try}$ block. At this point, the N parameter of the active effect construction is the continuation to resume the execution.

[1] *Deep handlers*, which are reinstalled after an effect is raised, can be implemented with shallow handlers and recursion.

Figure 2 shows an example of a handler implementing a global state effect supporting `read` and `write` operations. The handler uses a state-passing style. The recursive function `go` takes a continuation k_0, a value r to pass to the continuation, and the current global state σ. It runs the continuation under a handler that expects raised effects to be pairs of the form (et, w), where et is a *tag* indicating whether the operation is a `read` or `write`. Based on the tag, it recursively calls `go` with the appropriate return value and updated state. If the tag does not match `read` or `write`, it re-raises the effect to allow composition with another handler for other effects.

```
run_state ≜ λmain init. go main () init
where go ≜ rec go k0 r σ.
             try k0 r with
               v k   ⇒ match v with
                         (read, ()) ⇒ go k σ σ
                       | (write, y) ⇒ go k () y
                       |     (et, w) ⇒ go k (do (et, w)) σ
             | ret v ⇒ v
```

Fig. 2. Handler for a global state effect.

In addition to effect handlers, FicusLang has two built-in primitive effects. The first is the **`pick`** expression, which non-deterministically evaluates to an arbitrary boolean. The second is **`observe`** v, which performs a labeled transition with the label given by the value v. These **`observe`** statements are used as a form of ghost code and underlie our support for prophecy variables as discussed in §5. It might seem that including these two effects as primitives contradicts the claim of being able to bootstrap everything from effect handlers. In §E we present an alternative core calculus with these two effects removed and show that they too can be encoded in terms of effect handlers. However, in the body of this paper, we include these primitive effects as they are relatively simple.

2.2 Core Ficus Logic

To reason about programs written in FicusLang, we make use of Ficus, a separation logic built on top of the Iris framework [31], adapted from the Hazel logic for effect handlers [19]. This section first presents the basic core of Ficus, which is essentially a subset of Hazel. Later sections will describe additional generalizations that go beyond Hazel.

Ficus uses a weakest precondition assertion of the form ewp $e\ \langle\Psi\rangle\ \{\Phi\}$ for reasoning about programs. In this assertion, e is a program expression, Φ is a postcondition, and Ψ is a *protocol* that describes the specifications for effect handlers that are active as e executes. This assertion says that if e executes in an environment with handlers satisfying Ψ, then evaluating e will not get stuck, and if e terminates with value v, then the assertion $\Phi(v)$ will hold. More concretely, the protocol Ψ is a predicate of type $Val \to (Val \to iProp) \to iProp$, where the first argument is the value of the effect, and the second argument is the postcondition at the time the effect was raised. Throughout this paper, we require all protocols to be *monotonic* [18]. A protocol Ψ is monotonic if $(\forall w.\ \Phi(w) \mathrel{-\!\!*} \Phi'(w)) \vdash \Psi(v, \Phi) \mathrel{-\!\!*} \Psi(v, \Phi')$ for all v, Φ, and Φ'. This essentially enforces a one-shot continuation discipline in the logic which simplifies our presentation and suffices for our purposes. We will further discuss this design choice in §8.1.

Figure 3 lists a selection of reasoning rules for the ewp assertion. Most of these rules are similar to standard weakest precondition rules in separation logics. The key rule for reasoning about effects is Ewp-Do, which says that to raise value v, it suffices to show that the protocol holds for the value v and the current postcondition Φ.

For example, for the state handler in Figure 2, we use the STATE protocol

$$\begin{aligned}
\mathsf{READ}^{\gamma}(v, \Phi) &\triangleq \exists x.\ v = (\texttt{read}, ()) * S^{\gamma}(x) * (S^{\gamma}(x) \mathrel{-\!\!*} \Phi(x)) \\
\mathsf{WRITE}^{\gamma}(v, \Phi) &\triangleq \exists x, y.\ v = (\texttt{write}, y) * S^{\gamma}(x) * (S^{\gamma}(y) \mathrel{-\!\!*} \Phi(())) \\
\mathsf{STATE}^{\gamma}(v, \Phi) &\triangleq \mathsf{READ}^{\gamma}(v, \Phi) \vee \mathsf{WRITE}^{\gamma}(v, \Phi)
\end{aligned}$$

$$\textsc{Ewp-Value}\ \frac{\Phi(v)}{\mathsf{ewp}\ v\ \langle\Psi\rangle\ \{\Phi\}}\qquad \textsc{Ewp-Do}\ \frac{\Psi(v,\Phi)}{\mathsf{ewp}\ \mathbf{do}\ v\ \langle\Psi\rangle\ \{\Phi\}}\qquad \textsc{Ewp-Mono}\ \frac{\Psi \sqsubseteq \Psi' \qquad \forall v.\, \Phi(v) \mathrel{-\!\!*} \Phi'(v) \qquad \mathsf{ewp}\ e\ \langle\Psi\rangle\ \{\Phi\}}{\mathsf{ewp}\ e\ \langle\Psi'\rangle\ \{\Phi'\}}$$

$$\textsc{Ewp-Frame}\ \frac{R \qquad \mathsf{ewp}\ e\ \langle\Psi\rangle\ \{\Phi\}}{\mathsf{ewp}\ e\ \langle\Psi\rangle\ \{v.\, R * \Phi(v)\}}\qquad \textsc{Ewp-Step}\ \frac{\forall e'.\, e \rightarrow e' \mathrel{-\!\!*} \mathsf{ewp}\ e'\ \langle\Psi\rangle\ \{\Phi\} \qquad \exists e'.\, e \rightarrow e'}{\mathsf{ewp}\ e\ \langle\Psi\rangle\ \{\Phi\}}$$

$$\textsc{Ewp-Bind}\ \frac{\mathsf{ewp}\ e\ \langle\Psi\rangle\ \{v.\, \mathsf{ewp}\ N[v]\ \langle\Psi\rangle\ \{\Phi\}\}}{\mathsf{ewp}\ N[e]\ \langle\Psi\rangle\ \{\Phi\}}$$

Fig. 3. Selected reasoning rules about the effect weakest precondition $\mathsf{ewp}\ e\ \langle\Psi\rangle\ \{\Phi\}$.

where $S^\gamma(x)$ is a predicate (defined later) that uses a piece of *ghost state* with the name γ to assert that the current value of the global state σ is x. The first component of the protocol is READ, which says that when the effect tag is `read`, then the client must show $S^\gamma(x)$ for some x, in which case the protocol gives back $S^\gamma(x)$ for proving the postcondition Φ instantiated with the value x, indicating that the return value of the effect will be x. The second component is the WRITE protocol, which updates the given S^γ predicate from value x to the value y being written and returns back the unit value. Finally, STATE is the disjunction of these two protocols.

By applying Ewp-Do, we obtain the following derived rules for reasoning with this protocol:

$$\textsc{Ewp-Read}\ \frac{S^\gamma(x)}{\mathsf{ewp}\ \mathbf{do}\ (\mathtt{read}, ())\ \langle \mathsf{STATE}^\gamma\rangle\ \{v.\, v = x * S^\gamma(x)\}}\qquad \textsc{Ewp-Write}\ \frac{S^\gamma(x)}{\mathsf{ewp}\ \mathbf{do}\ (\mathtt{write}, y)\ \langle \mathsf{STATE}^\gamma\rangle\ \{v.\, v = () * S^\gamma(y)\}}$$

Installing Handlers. So far, we have seen how a client can reason about effects when an appropriate protocol is part of the ewp assertion. Protocols are added to the ewp when a handler is installed using the Ewp-Try rule shown below. Using the Ficus approach, we think of the language and logic as being extended to support new effects by adding handlers, so applying this rule forms the core proof obligation of a developer trying to extend the program logic.

$$\textsc{Ewp-Try}\ \frac{\mathsf{ewp}\ e\ \langle\Psi\rangle\ \{\Phi\} \qquad \begin{array}{l}\big(\forall v_2.\, \Phi(v_2) \mathrel{-\!\!*} \mathsf{ewp}\ e_2\ \langle\Psi'\rangle\ \{\Phi'\}\big) \wedge \\ \big(\forall v_1, k_1.\, \Psi(v_1, \lambda w.\, \mathsf{ewp}\ k_1\ w\ \langle\Psi\rangle\ \{\Phi\}) \mathrel{-\!\!*} \mathsf{ewp}\ e_1\ \langle\Psi'\rangle\ \{\Phi'\}\big)\end{array}}{\mathsf{ewp}\ \mathbf{try}\ e\ \mathbf{with}\ v_1\ k_1 \Rightarrow\ e_1 \mid \mathbf{ret}\ v_2 \Rightarrow e_2\ \langle\Psi'\rangle\ \{\Phi'\}}$$

In the Ewp-Try rule, below the line, we start with a protocol Ψ', and in the first premise above the line, we reason about the expression e, which runs under the new handler, with the protocol Ψ. This premise will be proved by a client who may reason as if e has access to the new effects.

Meanwhile, the second premise makes up the proof obligation that justifies extending the logic with this new protocol Ψ. This premise is a logical conjunction with two parts. The first conjunct is for the case where e evaluates to a value without raising an effect and requires showing that e's postcondition Φ implies an ewp about the remaining expression e_2. The second conjunct is for the case where e raises an effect. Recall that when e raises an effect, from the client's perspective it must establish the protocol Ψ. Conversely, that means that here in this premise, we get the protocol Ψ instantiated with the value v_1 raised with the effect, and a predicate that captures a specification for the continuation k_1. From this, we must prove an ewp about the handler code e_1 that will run.

These two conjuncts are joined with $\wedge$ instead of $*$ because only one of these two outcomes will occur, so the rule does not require separate resources for each conjunct.

To apply this rule for the state handler from Figure 2, and thereby add the STATE protocol, we first need to more carefully define the $S^\gamma(x)$ assertion. To do so, we use the underlying Iris logic's support for defining ghost state using *resource algebras* (RAs) [31]. In particular, we define it as the *fragment* copy of an (exclusive) *authoritative* RA: $S^\gamma(x) \triangleq \boxed{\circ x}^\gamma$. This RA comes with two types of ghost resources: an authoritative copy $\boxed{\bullet x}^\gamma$ and a fragment copy $\boxed{\circ x}^\gamma$. When these copies are combined, they are guaranteed to agree, *i.e.,* $\boxed{\bullet x}^\gamma * \boxed{\circ y}^\gamma \vdash x = y$, and they can be updated to an arbitrary value y, with the rule $\boxed{\bullet x}^\gamma * \boxed{\circ x}^\gamma \vdash \Rrightarrow \boxed{\bullet y}^\gamma * \boxed{\circ y}^\gamma$, where $\Rrightarrow$ is the *basic update modality*. The update modality is the primitive for manipulating ghost resources in the Iris logic. The assertion $\Rrightarrow P$ says that we can update our ghost resources and obtain P. The modality can be eliminated at any suitable time during program verification.

The handler $\texttt{run}_{\texttt{state}}$ allocates ghost states $\boxed{\bullet \mathit{init}}^\gamma$ and $\boxed{\circ \mathit{init}}^\gamma$ at a fresh ghost name γ, where *init* is the initial value for the state that is passed in. It keeps its authoritative copy $\boxed{\bullet \mathit{init}}^\gamma$, and passes the fragment copy $\boxed{\circ \mathit{init}}^\gamma$, which is $S^\gamma(\mathit{init})$, to the client. Whenever the client raises an effect, it must show the value satisfies STATE^γ, which is then passed to the handler code. The handler proof uses the $S^\gamma(x)$ that is included in STATE^γ and combines it with its corresponding authoritative copy of the ghost state to carry out the read or write. Note that the handler recursively calls go, thereby reinstalling the handler and running the continuation. To reason about this recursion, we use Löb induction from the underlying Iris logic [31, 37]. Altogether, we obtain the following derived rules for go and run, starting from a base protocol $\bot$ defined by $\bot(v, \Phi) \triangleq \mathsf{False}$.

$$\textsc{Ewp-StateGo}\quad \frac{\boxed{\bullet\sigma}^\gamma \qquad \mathsf{ewp}\ k\ r\ \langle \mathsf{STATE}^\gamma \rangle\ \{\Phi\}}{\mathsf{ewp}\ \texttt{go}\ k\ r\ \sigma\ \langle \bot \rangle\ \{\Phi\}}* \qquad \textsc{Ewp-StateRun}\quad \frac{\forall \gamma.\ S^\gamma(\mathit{init}) \mathrel{-\!\!*} \mathsf{ewp}\ \mathit{main}\ ()\ \langle \mathsf{STATE}^\gamma \rangle\ \{\Phi\}}{\mathsf{ewp}\ \texttt{run}_{\texttt{state}}\ \mathit{main}\ \mathit{init}\ \langle \bot \rangle\ \{\Phi\}}*$$

Building a Hierarchy of Effects. The previous example showed how to go from no effects (represented by protocol $\bot$) to the state effect (protocol STATE^γ). In practice, we want to accumulate effects by nesting additional handlers within the handler for STATE^γ. To that end, as in Hazel, we define a combinator $\oplus$ on protocols by $(\Psi_1 \oplus \Psi_2)(v, \Phi) \triangleq \Psi_1(v, \Phi) \vee \Psi_2(v, \Phi)$. In other words, $\Psi_1 \oplus \Psi_2$ represents that a client may choose to use effects from either Ψ_1 or Ψ_2, or both. For example, $\mathsf{STATE}^\gamma = \mathsf{READ}^\gamma \oplus \mathsf{WRITE}^\gamma$. Applying this operation to protocols results in a "larger" protocol. This is formally captured by a preorder relation on protocols $\Psi_1 \sqsubseteq \Psi_1 \oplus \Psi_2$. Intuitively, $\Psi_1 \oplus \Psi_2$ is larger than Ψ_1 because the former permits the client to raise more kinds of effects. Using the Ewp-Mono rule, we generalize Ewp-Read and Ewp-Write accordingly: rather than requiring exactly the protocol STATE^γ, we require that the protocol is some Ψ such that $\mathsf{STATE}^\gamma \sqsubseteq \Psi$.

Similarly, the Ewp-StateGo and Ewp-StateRun specifications for installing the handler do not need to start from the base protocol $\bot$. Instead, they can start from an arbitrary protocol Ψ, and—so long as Ψ does not already handle the tags for read and write—the client code would then operate with protocol $\Psi \oplus \mathsf{STATE}^\gamma$. This enables the state handler to be composed with an arbitrary context of previously installed handlers, allowing a logic developer to mixin rules for state with other effects. Formally, a protocol Ψ is said to *only handle* a set of tags T if $\Psi(v, \Phi) \vdash \exists t, w.\ v = (t, w) \wedge t \in T$. We write $\mathsf{tags}(\Psi)$ for the smallest set T such that Ψ only handles T. When installing the state protocol, we require $\texttt{read}, \texttt{write} \notin \mathsf{tags}(\Psi)$ for the protocol Ψ prior to installation. As another example of effects, we have implemented a handler for a heap effect supporting dynamically allocated higher-order references, with rules for local reads and writes. This handler uses the STATE protocol to store a global map representing the heap, and provides its own HEAP protocol for clients to use.

$$\mathsf{HEAP} \triangleq \mathsf{ALLOC} \oplus \mathsf{FREE} \oplus \mathsf{LOAD} \oplus \mathsf{STORE}$$

$$\mathsf{ALLOC}(v,\Phi) \triangleq \exists x.\, v = (\texttt{alloc}, x) * (\forall \ell.\, \ell \mapsto x \mathrel{-\!\!*} \Phi(\ell))$$
$$\mathsf{FREE}(v,\Phi) \triangleq \exists \ell, x.\, v = (\texttt{free}, \ell) * \ell \mapsto x * \Phi(())$$
$$\mathsf{LOAD}(v,\Phi) \triangleq \exists \ell, q, x.\, v = (\texttt{load}, \ell) * \ell \overset{q}{\mapsto} x * (\ell \overset{q}{\mapsto} x \mathrel{-\!\!*} \Phi(x))$$
$$\mathsf{STORE}(v,\Phi) \triangleq \exists \ell, x, y.\, v = (\texttt{store}, (\ell, y)) * \ell \mapsto x * (\ell \mapsto y \mathrel{-\!\!*} \Phi(()))$$
$$\mathbf{ref}\ v \triangleq \mathbf{do}\ (\texttt{alloc}, v) \qquad !\ell \triangleq \mathbf{do}\ (\texttt{load}, \ell) \qquad \ell \leftarrow v \triangleq \mathbf{do}\ (\texttt{store}, (\ell, v))$$

The *fractional* points-to connective $\ell \overset{q}{\mapsto} v$ [12] (with $\ell \mapsto v$ short for $\ell \overset{1}{\mapsto} v$) is just a piece of ghost state. It also depends on a ghost name γ_{heap}, but since γ_{heap} is typically globally fixed, we omit this name when the language under consideration only has one heap. The HEAP protocol provides the standard separation logic interface for the heap, matching the non-atomic heap laws of the HeapLang logic, the example program logic distributed with Iris.[2]

In this way, we compositionally build logical support for a collection of effects starting from the $\bot$ protocol, extending the core pure logic with support for these effects. This approach is grounded in an adequacy theorem, which shows that the logic starting with the $\bot$ protocol is sound.

Theorem 2.1 (Adequacy, Core Ficus). *Let φ be a first-order predicate (i.e., a Rocq* `Prop`*). If $\vdash \mathsf{ewp}\ e\ \langle \bot \rangle\ \{\varphi\}$, then executing e never gets stuck, and if $e \to^* v$ then $\varphi(v)$ holds.*

As in many other works on program logic, this theorem only considers *partial correctness*, *i.e.,* it is a *safety* property and it does not guarantee that the program will terminate.

3 Concurrency and Extensible Worlds

In the effects we have seen so far, the handler always immediately returns control to the client that raised the effect. However, for other kinds of effects, the handler may instead pass control to other client code. To help reason about these kinds of handlers, Ficus uses a new feature called *extensible worlds*.

A key example of an effect where this mechanism is useful is preemptive concurrency. Figure 4 shows an implementation of a concurrency handler. It represents the collection of threads as a bag (*i.e.,* multiset) called *pool*, and uses a function choose that non-deterministically removes an element from the bag and returns it along with the remaining bag. Each thread in *pool* is represented by a triple of (continuation, result of last effect, thread type), where the thread type can be either a main thread $\mathcal{M}$, a child thread $\mathcal{C}$, or their terminated variants $\mathcal{M}^\times$ and $\mathcal{C}^\times$.

```
run_conc ≜ λmain. go {|(main, (), M)|}
where go ≜ rec go pool.
  let ((k_0, r, t), pool) := choose pool in
  try k_0 r with
    v k    ⇒ match v with
               (fork, e) ⇒ go ({|(e, (), C), (k, (), t)|} ⊎ pool)
             | (et, w)   ⇒ go ({|(k, do (et, w), t)|} ⊎ pool)
  | ret v ⇒ if t = M^× then v
            else go ({|((λ_. v), (), t^×)|} ⊎ pool)
```

Fig. 4. A handler for concurrency.

At each iteration, the scheduler non-deterministically chooses one thread from *pool* and lets it run for as many pure steps as it can until it raises an effect or terminates. If the thread raises a fork effect, the scheduler will push both the new thread $(e, (), C)$ and the old thread $(k, (), t)$ to the thread pool. If the thread raises another type of effect, the scheduler will forward the effect to an outer handler by re-raising it, collect its result, and put the old thread back in the thread pool. Finally, if the thread terminates, the scheduler will mark the thread as terminated and put it back into *pool*. If choose later selects the terminated main thread $\mathcal{M}^\times$, the scheduler stops and returns the main thread's final value v. Thus, other threads may keep running for a while even after the main thread

[2]Here we omit from ALLOC the Iris "meta token", which associates ghost state with a location.

has terminated, which corresponds to the operational semantics of the HeapLang language. Note that the scheduler itself is agnostic to outer handlers, and in particular, it makes no assumption on the communication model between threads, which is implemented by outer handlers.

Because the handler may pass control to other threads when an effect is raised, we now need to reason about coordination between threads, which is what extensible worlds will enable.

3.1 Background: Iris Invariants and Fancy Updates

To motivate extensible worlds, let us first recall how modern concurrent separation logics (CSLs) like Iris handle reasoning about interaction between different threads. By default, CSL allows for local reasoning about different threads in a concurrent system by dividing up state and resources into separate disjoint parts using separating conjunction, with each thread having ownership of some portion of the state. However, in some cases, threads need to *share* ownership of state. To do so, CSLs make use of *invariants*. In Iris, an invariant assertion $\boxed{P}^{\mathcal{N}}$ says that P is an invariant that holds between all program steps. The $\mathcal{N}$ annotation is a *name* given to this invariant. These assertions are *persistent*, ensuring that $\boxed{P}^{\mathcal{N}} \vdash \boxed{P}^{\mathcal{N}} * \boxed{P}^{\mathcal{N}}$, so each thread can have its own copy of the assertion. When carrying out a proof about a thread, we access the underlying assertion P by "opening" the invariant using the following rule:

$$\textsc{Wp-InvAcc}\quad \frac{\boxed{P}^{\mathcal{N}} \qquad \mathcal{N} \subseteq \mathcal{E} \qquad \triangleright P \mathrel{-\!\!*} \mathsf{wp}_{\mathcal{E}\setminus\mathcal{N}}\, e\, \{x.\ \triangleright P * \Phi(x)\} \qquad \mathsf{atomic}(e)}{\mathsf{wp}_{\mathcal{E}}\, e\, \{\Phi\}}$$

The third premise of this rule has us prove the weakest precondition under the assumption that P holds (guarded by a later modality $\triangleright$ [4, 11, 42], which we ignore for now), so long as we reestablish P in the postcondition of e. Here, e must be atomic, meaning it reduces to a value in a single step, so that by reestablishing P in the postcondition, we ensure that P will continue to hold before and after each step. The *mask* $\mathcal{E}$ is a set that tracks invariants that have not yet been opened. The invariants in $\mathcal{E}$ are said to be *closed* or *enabled*, while all other invariants are *open* or *disabled*.

In fact, in Iris, Wp-InvAcc is a derived rule. Iris uses a more primitive mechanism called a *fancy update modality* of the form ${}_{\mathcal{E}_1}\!\Rrightarrow_{\mathcal{E}_2}$ that encodes the process of opening and closing invariants. Informally, the assertion ${}_{\mathcal{E}_1}\!\Rrightarrow_{\mathcal{E}_2} P$ states that starting with all invariants in $\mathcal{E}_1$ being enabled, and then opening/closing invariants so as to end up with $\mathcal{E}_2$ being enabled, it is possible to prove P. Then the Wp-InvAcc rule can be derived from the following two rules.

$$\textsc{Fupd-InvAcc}\quad \frac{\boxed{P}^{\mathcal{N}} \qquad \mathcal{N} \subseteq \mathcal{E}}{\mathcal{E} \Rrightarrow \mathcal{E}\setminus\mathcal{N} \triangleright P * (\triangleright P \mathrel{-\!\!*} {}_{\mathcal{E}\setminus\mathcal{N}}\!\Rrightarrow_{\mathcal{E}} \mathsf{True})} \qquad \textsc{Wp-Atomic}\quad \frac{{}_{\mathcal{E}_1}\!\Rrightarrow_{\mathcal{E}_2} \mathsf{wp}_{\mathcal{E}_2}\, e\, \{x.\ {}_{\mathcal{E}_2}\!\Rrightarrow_{\mathcal{E}_1} \Phi(x)\} \qquad \mathsf{atomic}(e)}{\mathsf{wp}_{\mathcal{E}_1}\, e\, \{\Phi\}}$$

These rules are notationally heavy, but the rule on the left captures the process of opening an invariant with the update modality. Starting from masks in $\mathcal{E}$, we end up with masks in $\mathcal{E} \setminus \mathcal{N}$, and get $\triangleright P$. Additionally, we get that by supplying $\triangleright P$ we can close the invariant, as represented by the ${}_{\mathcal{E}\setminus\mathcal{N}}\!\Rrightarrow_{\mathcal{E}} \mathsf{True}$. Meanwhile, the rule on the right is what allows us to actually use the fancy update modality to open invariants when reasoning about an atomic expression e, so long as the postcondition also includes a modality to close those same invariants.

Under the hood, the semantic model for this ${}_{\mathcal{E}_1}\!\Rrightarrow_{\mathcal{E}_2}$ modality uses a mechanism called *world satisfaction*. Essentially, the Iris definition of ${}_{\mathcal{E}_1}\!\Rrightarrow_{\mathcal{E}_2}$ tracks the set of all of the enabled/disabled invariants, and requires that for each enabled invariant, there are resources ensuring the invariant holds. This bundle of resources is called a world.

Although the Iris invariant mechanism is very expressive and flexible, it has some limitations. As a result, some prior projects have found it necessary to modify this notion of invariant. For example, both Perennial [13] and Nola [39] have considered alternate forms of invariant assertions, the former to encode invariants that govern behavior when a program crashes, and the latter to reason about termination without needing the later modality. One key issue, for our purposes, is that the notion of atomicity and the way invariants can be used in a rule like Wp-Atomic are closely tied to the built-in preemptive concurrency in Iris. Instead, we want to allow handler implementers to define a notion of invariant suitable for the kind of effect they are modeling.

3.2 Extensible Worlds

To achieve this kind of extensibility, Ficus does not fix a single baked-in world in the interpretation of the fancy update. Instead, Ficus parameterizes the ewp and fancy update modalities by a customizable notion of world. The full version of the Ficus ewp assertion then has the form $\mathsf{ewp}_{\mathrm{W}_1,\mathrm{W}_2}\ e\ \langle\Psi\rangle\ \{\Phi\}$, where W_1 is an arbitrary Iris proposition representing the world at the start of e's execution, and W_2 is the world after e finishes executing. Meanwhile, the fancy update modality becomes the *world update modality*, defined as ${}_{\mathrm{W}_1}\Rightarrow_{\mathrm{W}_2} P \triangleq \mathrm{W}_1 \mathrel{-\!\!*} \Rightarrow (P * \mathrm{W}_2)$, which intuitively states that given access to the initial world W_1, we can prove P and the resulting world W_2, potentially performing ghost updates using the basic update modality. When the starting world W is the same as the ending world, we simply write $\mathsf{ewp}_{\mathrm{W}}\ e\ \langle\Psi\rangle\ \{\Phi\}$ and $\Rightarrow_{\mathrm{W}}$.

Worlds are just normal Iris assertions, but it is nevertheless helpful to think of them more abstractly. The combination of two worlds, written $\mathrm{W}_1 \oplus \mathrm{W}_2$, is defined as $\mathrm{W}_1 * \mathrm{W}_2$. We impose a preorder $\sqsubseteq$ on worlds defined by $\mathrm{W}_1 \sqsubseteq \mathrm{W}_2 \triangleq \exists \mathrm{W}'.\ (\mathrm{W}_2 \dashv\vdash \mathrm{W}_1 \oplus \mathrm{W}')$. Here, larger worlds have more resources, and the minimal element $\bot$ is the proposition True. Thus, with an update like ${}_{\mathrm{W}_1}\Rightarrow_{\mathrm{W}_2} P$, when $\mathrm{W}_2 \sqsubseteq \mathrm{W}_1$, we are shifting to a *smaller* world, and give the difference between W_2 and W_1 to the proof of P. Conversely, shifting to a larger world with $\mathrm{W}_1 \sqsubseteq \mathrm{W}_2$ requires putting in the difference between W_1 and W_2.

The rules we have seen previously for the ewp are generalized to account for worlds. Selected generalized rules about ewp and the world update modality are shown in Figure 5. Wupd-Elim eliminates a world update modality from an assumption, updating the worlds on the goal accordingly. Wupd-Intro introduces a world update ${}_{\mathrm{W}_1}\Rightarrow_{\mathrm{W}_2} P$ from a larger world into a smaller world and gives a token ${}_{\mathrm{W}_2}\Rightarrow_{\mathrm{W}_1}$ True to restore the original world. The Wupd-Frame allows for "framing out" an unnecessary world W that occurs in both the starting and ending world.

Unlike the Iris Wp-Atomic rule, which only allows masks to change around an atomic step, the Ewp-WupdPre rule allows us to apply a world update that changes the starting world for any expression, and Ewp-WupdPost changes the corresponding ending world. This is allowed because, unlike standard Iris, where the scheduler could preempt a thread at any point, in the effect handler approach, control can only be transferred when an effect is raised. This means the starting world does not need to be immediately restored. Instead, only when an effect is raised, must the world be in an appropriate configuration, depending on whether the protocol Ψ requires it or not. In Ewp-DoWupd, we start by shifting to the bottom world $\bot$, and then in the continuation passed to the protocol Ψ, we must restore W_2. Finally, we can frame out an unused world in ewp with Ewp-WorldFrame.

Recovering Iris Invariants. It is straightforward to recover Iris-style impredicative invariants and the Iris fancy update modality in this more general world setting. As was described above, the standard Iris definition fixes some particular world in its definition of fancy updates, and uses ghost state to track the enabled invariants. Let us write $\mathsf{Tokl}(\mathcal{E})$ for the assertion that bundles the

$$\textsc{Wupd-Elim}\ \frac{Q \vdash {}_{W_2}\Rightarrow_{W_3} P}{({}_{W_1}\Rightarrow_{W_2} Q) \vdash {}_{W_1}\Rightarrow_{W_3} P} \qquad \textsc{Wupd-Intro}\ \frac{({}_{W_2}\Rightarrow_{W_1} \mathsf{True}) \mathrel{-\!\!*} P \quad W_2 \sqsubseteq W_1}{{}_{W_1}\Rightarrow_{W_2} P}* \qquad \textsc{Wupd-Frame}\ \frac{{}_{W_1}\Rightarrow_{W_2} Q}{{}_{W_1 \oplus W}\Rightarrow_{W_2 \oplus W} Q}*$$

$$\textsc{Ewp-ValueWupd}\ \frac{{}_{W_1}\Rightarrow_{W_2} \Phi(v)}{\mathsf{ewp}_{W_1,W_2}\, v\, \langle\Psi\rangle\, \{\Phi\}}* \qquad \textsc{Ewp-WupdPre}\ \frac{{}_{W_1}\Rightarrow_{W_2} \mathsf{ewp}_{W_2,W_3}\, e\, \langle\Psi\rangle\, \{\Phi\}}{\mathsf{ewp}_{W_1,W_3}\, e\, \langle\Psi\rangle\, \{\Phi\}}* \qquad \textsc{Ewp-WupdPost}\ \frac{\mathsf{ewp}_{W_1,W_2}\, e\, \langle\Psi\rangle\, \{v.\ {}_{W_2}\Rightarrow_{W_3} \Phi(v)\}}{\mathsf{ewp}_{W_1,W_3}\, e\, \langle\Psi\rangle\, \{\Phi\}}*$$

$$\textsc{Ewp-DoWupd}\ \frac{{}_{W_1}\Rightarrow_{\bot} \Psi(v, (\lambda r.\ {}_{\bot}\Rightarrow_{W_2} \Phi(r)))}{\mathsf{ewp}_{W_1,W_2}\ \mathbf{do}\ v\, \langle\Psi\rangle\, \{\Phi\}}* \qquad \textsc{Ewp-WorldFrame}\ \frac{\mathsf{ewp}_{W_1,W_2}\, e\, \langle\Psi\rangle\, \{\Phi\}}{\mathsf{ewp}_{W_1 \oplus W, W_2 \oplus W}\, e\, \langle\Psi\rangle\, \{\Phi\}}*$$

Fig. 5. Selected reasoning rules for ${}_{W_1}\Rightarrow_{W_2}$ and ewp with worlds.

world with the ghost state saying that mask $\mathcal{E}$ is enabled.[3] Then we recover the following analogue of the Fupd-InvAcc rule that we saw earlier.

$$\textsc{Wupd-InvAcc}\ \frac{\boxed{P}^{\mathcal{N}} \quad \mathcal{N} \subseteq \mathcal{E}}{{}_{\mathsf{TokI}(\mathcal{E})}\Rightarrow_{\mathsf{TokI}(\mathcal{E}\setminus\mathcal{N})} \triangleright P * (\triangleright P \mathrel{-\!\!*} {}_{\mathsf{TokI}(\mathcal{E}\setminus\mathcal{N})}\Rightarrow_{\mathsf{TokI}(\mathcal{E})} \mathsf{True})}*$$

Moreover, by combining this rule with Ewp-WorldFrame, we can support Iris invariants while including other possible components in the world. As we will see in §6.1, this allows us to encode a mechanism similar to Perennial's crash borrows [56] while retaining standard Iris invariants.

3.3 Protocol for the Concurrency Handler

Now that we have an extensible mechanism for encoding invariants that hold across threads, we turn to the protocol for the concurrency handler.

One challenge is that the concurrency handler in Figure 4 is generic, in the sense that it does not know about the other effects that might be supported by outer handlers. It simply re-raises those effects to the outer handler and potentially transfers control to another thread when the effect returns. This means that the outer handlers could, say, implement shared memory or channel-based message passing concurrency, or some combination thereof. Ideally, the protocol we develop should work for different kinds of outer effects.

To achieve this, the first ingredient is a *protocol transformer* ATOM_W that lifts a protocol for these outer effects into a concurrent protocol, where W is a world that describes shared resources that can be accessed by different threads. To do this lifting, ATOM transforms Ψ to ensure that as part of raising an effect governed by Ψ, the thread must be able to restore the world W. In addition, when the handler returns control to a thread, it promises that W will hold. Formally, this is captured through the following definition

$$\mathsf{ATOM}_W(\Psi)(v, \Phi) \triangleq \Psi(v, \lambda r.\ {}_{\bot}\Rightarrow_W {}_W\Rightarrow_{\bot} \Phi(r))$$

The first ${}_{\bot}\Rightarrow_W$ is an obligation for the thread raising the effect to establish W after the effect completes. Meanwhile, because the second ${}_W\Rightarrow_{\bot}$ precedes the continuation $\Phi(r)$, it effectively gives back access to W before the continuation's Φ must be proved.

To better understand this definition, let's recall that the intuitive reading of ${}_{W_1}\Rightarrow_{W_2} P$ as a proof obligation is that we are currently in world W_1, and we need to perform an update to the world W_2 and prove P. If W_1 is smaller than W_2, we need to prove the difference between W_2 and W_1;

[3] Here the letter "I" stands for "invariant", so the whole notation means a token for invariants.

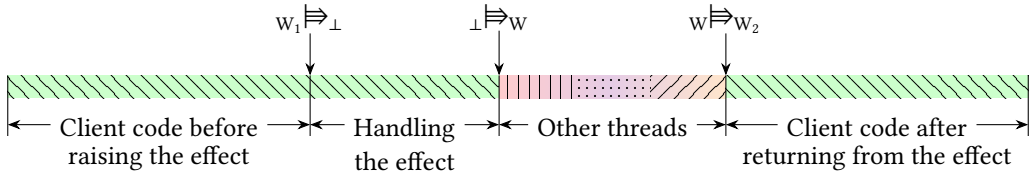


Fig. 6. Interleaving of threads under the $\mathsf{ATOM}_W(\Psi)$ protocol and the use of its world updates.

conversely, when W_1 is larger than W_2, we obtain the difference between W_1 and W_2. With this in mind, let us examine what the client needs to prove to raise an effect under the $\mathsf{ATOM}_W(\Psi)$ protocol. Using Ewp-DoWupd, we derive the following rule:

$$\frac{{}_{W_1}\Rrightarrow_{\bot} \Psi(v, (\lambda r.\ {}_{\bot}\Rrightarrow_{W} {}_{W}\Rrightarrow_{\bot} {}_{\bot}\Rrightarrow_{W_2} \Phi(r)))}{\mathsf{ewp}_{W_1,W_2}\ \mathbf{do}\ v\ \langle \mathsf{ATOM}_W(\Psi)\rangle\ \{\Phi\}}*$$

Here it might be helpful to think of W_1 as "the local world" before the effect, W as the "shared world" (for example, containing all the invariants that all threads rely on), and W_2 as the local world after the effect has returned. In practice, W is typically larger than or equal to W_1/W_2. The obligation above the line is equivalent to

$${}_{W_1}\Rrightarrow_{\bot} \Psi(v, (\lambda r.\ {}_{\bot}\Rrightarrow_{W} {}_{W}\Rrightarrow_{W_2} \Phi(r)))$$

The three updates here are used at different places, as depicted in Figure 6.

- ${}_{W_1}\Rrightarrow_{\bot}$ is used before raising an effect. Since it is an update to $\bot$, the client obtains everything in W_1, for example, it can open any invariants.
- ${}_{\bot}\Rrightarrow_{W}$ is used after the effect has been executed but right before the scheduler switches to another thread. It requires the client to establish everything in the shared world W, for example, closing all invariants. Since W can be larger than W_1, if some invariants were open before raising the effect, the client must also close them now.
- ${}_{W}\Rrightarrow_{W_2}$ is used after the execution of the initial thread resumes. Again, because W is larger than W_2, this update allows the client to open more invariants.

As a special case, if we instantiate W to be $\mathsf{TokI}(\top)$, where $\top$ is the full mask saying that all invariants are enabled, then the $\mathsf{ATOM}_W(\Psi)$ protocol requires a thread to close all Iris invariants after an effect completes, just as in the Iris rule Wp-Atomic. Under this protocol, we have an analogue of Wp-InvAcc, which relaxes condition $\mathsf{atomic}(e)$ to $\mathsf{eff\text{-}atomic}(e)$, *i.e.,* e consists of several pure steps and one effectful step (*i.e.,* a **do** operation).

Ewp-InvAcc

$$\frac{\boxed{P}^{\mathcal{N}} \qquad \mathcal{N} \subseteq \mathcal{E} \qquad \triangleright P \mathrel{-\!\!*} \mathsf{ewp}_{\mathsf{TokI}(\mathcal{E}\setminus\mathcal{N})}\ e\ \langle \mathsf{ATOM}_{\mathsf{TokI}(\top)}(\Psi)\rangle\ \{x.\ \triangleright P * \Phi(x)\} \qquad \mathsf{eff\text{-}atomic}(e)}{\mathsf{ewp}_{\mathsf{TokI}(\mathcal{E})}\ e\ \langle \mathsf{ATOM}_{\mathsf{TokI}(\top)}(\Psi)\rangle\ \{\Phi\}}*$$

Although being able to keep invariants open across multiple pure steps might not bring extra expressivity, it results in simpler proofs in some scenarios. For example, when an invariant contains pure facts needed to perform a pure step, Ewp-InvAcc lets us open that invariant once for both the pure step and a subsequent effectful step, whereas the standard Iris rule Wp-InvAcc requires opening it twice. Another benefit is that Ewp-InvAcc lets the pure steps of e be used to remove the later modality $\triangleright$ from P after opening the invariant. In Iris, this would otherwise require later credits [51], which not all Iris-based logics support.

To get the final protocol CONC for the concurrency handler, we combine ATOM with a protocol FORK governing the `fork` effect. Because the forked thread will run in the scope of the concurrency

handler, FORK must have a recursive dependence on CONC.

$$\mathsf{CONC}_{\mathrm{W}}(\Psi) \triangleq \mathsf{ATOM}_{\mathrm{W}}(\Psi \oplus \mathsf{FORK}_{\mathrm{W}}(\Psi))$$
$$\mathsf{FORK}_{\mathrm{W}}(\Psi)(v, \Phi) \triangleq \exists e.\, v = (\mathtt{fork}, \lambda_.\, e) * \triangleright\, \mathrm{ewp}_{\mathrm{W}}\, e\, \langle \mathsf{CONC}_{\mathrm{W}}(\Psi)\rangle\, \{_.\, \mathrm{True}\} * \Phi(())$$

Here, the recursive occurrence of CONC in FORK occurs under the later modality $\triangleright$, so that we can define the result as a *guarded fixed point* [16, 22, 31]. The FORK protocol requires showing an appropriate ewp for the forked thread. Let us introduce a wrapper $\mathbf{fork}\ e \triangleq \mathbf{do}\ (\mathtt{fork}, \lambda_.\, e)$. We can re-derive the standard **fork** rule from Iris with this protocol.

$$\frac{\textsc{Ewp-Fork}\quad \mathrm{ewp}_{\mathrm{W}}\, e\, \langle \mathsf{CONC}_{\mathrm{W}}(\Psi)\rangle\, \{_.\, \mathrm{True}\}}{\mathrm{ewp}_{\mathrm{W}}\, \mathbf{fork}\, e\, \langle \mathsf{CONC}_{\mathrm{W}}(\Psi)\rangle\, \{v.\, v = ()\}} *$$

Finally, we have the specification for $\mathtt{run}_{\mathsf{conc}}$ from Figure 4, which installs the concurrency handler.

$$\frac{\textsc{Ewp-ConcRun}\quad \mathrm{ewp}_{\mathrm{W}}\, \mathit{main}\, ()\, \langle \mathsf{CONC}_{\mathrm{W}}(\Psi)\rangle\, \{\Phi\} \qquad \mathtt{fork} \notin \mathrm{tags}(\Psi)}{\mathrm{ewp}_{\mathrm{W}}\, \mathtt{run}_{\mathsf{conc}}\, \mathit{main}\, \langle \Psi\rangle\, \{\Phi\}} *$$

In addition to the ability to create threads with **fork**, we can also model primitive atomic instructions such as compare-and-swap (CAS), atomic exchange (XCHG), or fetch-and-add (FAA). To do so, we just need to define an additional handler on top of the heap handler and associated protocol ATOMHEAP that models these effects, much like the earlier HEAP protocol did.

$$\mathsf{ATOMHEAP} \triangleq \mathsf{CAS} \oplus \mathsf{XCHG} \oplus \mathsf{FAA}$$
$$\mathsf{CAS}(v, \Phi) \triangleq \exists \ell, q, w, x, y.\, v = (\mathtt{cas}, (\ell, x, y)) * \ell \overset{q}{\mapsto} w * (q = 1 \vee w \neq x) * (\ell \overset{q}{\mapsto} (\text{if } w = x \text{ then } y \text{ else } w) \mathrel{-\!\!*} \Phi((w, w = x)))$$
$$\mathsf{XCHG}(v, \Phi) \triangleq \exists \ell, x, y.\, v = (\mathtt{xchg}, (\ell, y)) * \ell \mapsto x * (\ell \mapsto y \mathrel{-\!\!*} \Phi(x))$$
$$\mathsf{FAA}(v, \Phi) \triangleq \exists \ell, z_1, z_2.\, v = (\mathtt{faa}, (\ell, z_2)) * \ell \mapsto z_1 * (\ell \mapsto z_1 + z_2 \mathrel{-\!\!*} \Phi(z_1))$$

3.4 Recovering the HeapLang Logic

Combining HEAP, ATOMHEAP, and protocols for prophecy variables that will be introduced in §5, and applying the CONC protocol transformer, we obtain a protocol that models all of the operations one finds in HeapLang. In fact, an ewp under these protocols satisfies all reasoning rules that the HeapLang wp satisfies. Additionally, we have obtained a stronger version of the invariant opening rule, allowing us to keep invariants open for multiple pure steps.

To facilitate interactive proofs, our Rocq development includes tactics that provide a high-level proof experience similar to that of the HeapLang logic. Using these tactics, we have ported all programs and specifications in the standard library of HeapLang to Ficus, and a verification of the Herlihy-Wing queue [28]. The proofs of these specifications are identical to their HeapLang counterparts except that HeapLang tactics are substituted by corresponding Ficus tactics.

Discussion: Ficus Concurrency Model. A careful reader might object that what allows us to derive this stronger invariant opening rule is the fact that the concurrency handler only transfers control to another thread when an effect is raised. In contrast, a standard operational semantics for preemptive concurrency typically allows for preemption at *every* step, thereby generating more possible interleavings of thread operations. Because our handler semantics for concurrency is not generating all of the interleavings that the standard semantics would, one might wonder whether this handler adequately models concurrency. Fortunately, verifying programs against this handler

is still sound because, under partial correctness, the intermediate pure steps between effects are not observable to other threads in the system. Thus, omitting preemption points between these pure steps cannot hide any safety violations. In the next section, we introduce a *relational logic* that will allow us to prove this claim rigorously.

4 Contextual Equivalence of Effectful Programs

In this section, we develop RelFicus, a relational logic that allows us to prove correspondences between the behavior of two effectful programs written in FicusLang. Just as with unary reasoning, we compositionally derive relational reasoning rules for a number of effects. Using the resulting logic, we define a logical relations model that can prove contextual equivalences of programs written in typed subsets of FicusLang. We apply this logical relations model to prove that inserting additional preemption points in our concurrency handler does not change the set of possible program behaviors, thus justifying the semantics from the previous section where preemption only occurs when effects are raised.

4.1 Background: Embedding Relational Logics into Unary Logics

RelFicus embeds relational reasoning into Ficus by encoding a second program as *ghost state*, as in CaReSL [58]. Let us first recall how this ghost state encoding works in modern Iris-based separation logics. For sequential programs, one first introduces two assertions: $\mathsf{spec}(e)$, which says that the second program (which we call the "specification" or "spec" program) is currently represented by the expression e; and specCtx, which is an invariant that ensures that the $\mathsf{spec}(e)$ ghost state can only be updated in ways that represent valid transitions in the language's operational semantics. One needs to further add in assertions to represent the state of this spec program, for example, a spec points-to assertion $\ell \mapsto_s v$ says that in the spec program, location ℓ contains the value v, analogous to the standard points-to assertion. Using these assertions, one derives rules that allow for the spec program to be "executed" by applying ghost updates. To prove a relational property about two programs e_1 and e_2, it then suffices to derive a judgment of the form

$$\mathsf{specCtx} * \mathsf{spec}(e_2) \vdash \mathsf{wp}\ e_1\ \{v_1.\ \exists v_2.\ \mathsf{spec}(v_2) * \varphi(v_1, v_2)\}$$

The soundness theorem of the encoding says that such a derivation implies that, for every execution of e_1, if it terminates in a value v_1, then there exists a terminating execution of e_2 ending in some value v_2 such that $\varphi(v_1, v_2)$ holds. As usual for partial correctness, if an execution of e_1 does not terminate, the soundness theorem says nothing about e_2. This basic approach can be generalized to account for concurrency as well by having multiple spec program resources, one for each thread in the concurrent program.

RelFicus adapts this style of relational reasoning with ghost programs to the setting of effect handlers. A key challenge is that the usual ghost state encoding requires fixing the primitive effects of the language ahead of time, and requires special treatment in the concurrent case to introduce per-thread spec programs. In contrast, in RelFicus these notions are derivable using protocols and worlds, just as with unary reasoning.

4.2 RelFicus: A Relational Logic for FicusLang

To enable extensibility, RelFicus reasons about spec programs using an *effect specification resource* assertion $\mathsf{espec}_{\mathrm{W}}\ e\ \langle\Psi\rangle$ that tracks a spec program e and a protocol Ψ in a world W. The program e can be updated and progressed according to the operational semantics of FicusLang. For example, $\mathsf{espec}_{\mathrm{W}}\ ((\lambda x.\ e)\ v)\ \langle\Psi\rangle$ can be updated to $\mathsf{espec}_{\mathrm{W}}\ e[v/x]\ \langle\Psi\rangle$ to reflect the execution of a beta reduction as justified by Espec-Pure shown below. As in Ficus, the protocol Ψ describes the effect handlers that are active as e executes. That is, $\mathsf{espec}_{\mathrm{W}}\ N[\mathbf{do}\ v]\ \langle\Psi\rangle$ can be updated to

$\mathsf{espec}_W\ N[w]\ \langle\Psi\rangle$ for some w such that $\Phi(w)$ by establishing $\Psi(v, \Phi)$ as enabled by Espec-Do.

$$\mathsf{espec}_W\ K[e]\ \langle\Psi\rangle * e \rightarrow^* e' \vdash \Rrightarrow_W \mathsf{espec}_W\ K[e']\ \langle\Psi\rangle \qquad \textsc{Espec-Pure}$$

$$\mathsf{espec}_W\ N[\mathbf{do}\ v]\ \langle\Psi\rangle * \Psi(v, \Phi) \vdash \Rrightarrow_W \exists w.\ \mathsf{espec}_W\ N[w]\ \langle\Psi\rangle * \Phi(w) \qquad \textsc{Espec-Do}$$

Using these rules, we obtain derived rules for raising specific effects like global state, much as in the unary case in §2, for example,

$$\mathsf{espec}_W\ \mathbf{do}\ (\mathtt{read}, ())\ \langle\Psi \oplus \mathsf{STATE}^\gamma\rangle * S^\gamma(v) \vdash \Rrightarrow_W \mathsf{espec}_W\ v\ \langle\Psi \oplus \mathsf{STATE}^\gamma\rangle * S^\gamma(v)$$

$$\mathsf{espec}_W\ \mathbf{do}\ (\mathtt{write}, w)\ \langle\Psi \oplus \mathsf{STATE}^\gamma\rangle * S^\gamma(v) \vdash \Rrightarrow_W \mathsf{espec}_W\ ()\ \langle\Psi \oplus \mathsf{STATE}^\gamma\rangle * S^\gamma(w)$$

Similarly, we have rules for installing handlers that capture these effects, such as

$$\frac{\mathsf{espec}_W\ \mathtt{run}_{\mathtt{state}}\ \mathit{main}\ \mathit{init}\ \langle\Psi\rangle \qquad \mathtt{read}, \mathtt{write} \notin \mathsf{tags}(\Psi)}{\Rrightarrow_W \exists\gamma.\ S^\gamma(\mathit{init}) * \mathsf{espec}_W\ \mathit{main}\ ()\ \langle\Psi \oplus \mathsf{STATE}^\gamma\rangle}*$$

The following adequacy theorem for RelFicus holds, which requires that both the spec protocol and the ewp protocol are $\bot$.

Theorem 4.1 (Adequacy, RelFicus). *Let φ be a first-order relation. If*

$$\mathsf{espec}_\bot\ e_2\ \langle\bot\rangle \vdash \mathsf{ewp}_\bot\ e_1\ \langle\bot\rangle\ \{v_1.\ \exists v_2.\ \mathsf{espec}_\bot\ v_2\ \langle\bot\rangle * \varphi(v_1, v_2)\}$$

and $e_1 \rightarrow^* v_1$ then there exists a value v_2 such that $e_2 \rightarrow^* v_2$ and $\varphi(v_1, v_2)$.

The key to proving this adequacy theorem lies in coming up with a suitable definition of the $\mathsf{espec}_W\ e\ \langle\Psi\rangle$ resource that validates the above rules.

Constructing the Effect Specification Resource. Much like the ewp assertion, the specification resource $\mathsf{espec}_W\ e\ \langle\Psi\rangle$ tracks the behavior of the program e *under the assumption* that it executes in a program context satisfying the protocol Ψ and a logical world W. To define espec, we first define a more general construction genspec which we will specialize to obtain espec. The genspec assertion takes some abstract notion of a spec program and transforms it into a version that has protocols and worlds for reasoning about effects. Specifically, let $\mathsf{spec} : \mathit{Expr} \to \mathit{iProp}$ be any predicate that can be angelically updated according to the operational semantics: $\mathsf{spec}(e) \vdash \Rrightarrow_\bot \mathsf{spec}(e')$ as long as $e \rightarrow^* e'$. Given a choice of a spec predicate, the genspec assertion is defined as

$$\mathsf{genspec}_W\ e\ \langle\Psi\rangle \triangleq \exists K.\ \mathsf{spec}(K[e]) * \mathsf{handler}_W(\Psi, K)$$

where the assertion $\mathsf{handler}_W(\Psi, K)$ captures that K is an evaluation context that realizes the protocol Ψ indefinitely from the perspective of a program e running inside that context. Formally, this is expressed using a *greatest fixed point* on parameter K:[4]

$$\mathsf{handler}_W(\Psi, K) \triangleq \forall v.\ \mathsf{spec}(K[v]) \mathrel{-\!\!*} \Rrightarrow_W \mathsf{spec}(v) \tag{1}$$

$$\wedge\ \forall v, N, \Phi.\ \mathsf{spec}(K[\S(N)[v]]) * \Psi(v, \Phi) \mathrel{-\!\!*} \Rrightarrow_W \tag{2}$$

$$\exists K', w.\ \mathsf{spec}(K'[N[w]]) * \Phi(w) * \mathsf{handler}_W(\Psi, K')$$

The two conjuncts of this definition require that

(1) if e is a value v, then the context terminates with value v; and
(2) if e is a raised effect with continuation N and value v, *i.e.*, $e = \S(N)[v]$, and $\Psi(v, \Phi)$ holds, then N is reinstated with some value w in a context K' such that $\Phi(w)$ and $\mathsf{handler}_W(\Psi, K')$ holds co-recursively.

[4] A guarded fixed point, with the recursive occurrence under a $\triangleright$, would not suffice to derive Espec-Do: each unfolding would leave a $\triangleright$ that can only be removed when the implementation program takes a step, so the specification program could handle at most one effect per step of the implementation program.

We can derive generic versions of the Espec-Pure and Espec-Do rules for genspec, as well as examples like the STATE protocol. Then we obtain espec and specialized versions of these rules by instantiating genspec with $\mathrm{spec}(e) \triangleq e_0 \to^* e$, where e_0 is the initial expression that the ghost program starts as. Later, we will see how instantiating the definition with other choices of the base specification resource allows us to reason about thread-local effects in concurrent execution.

4.3 Concurrency

To reason relationally about effects that do not immediately transfer control back to the raising thread, we need to make use of extensible worlds, just as we did with the unary logic for concurrency. However, in the relational case, our specification of concurrency has an additional requirement: we want to be able to reason about each thread in the concurrent system individually. In the encoding of specification programs in CaReSL described above in §4.1, this is achieved by having a specification assertion per thread. Since each thread can raise effects and use other components of a protocol, we need these per-thread resources to have access to the protocol, just as in espec.

To construct this per-thread effect specification resource, we again use the genspec construction. To do so, we first need an underlying per-thread local specification resource $\mathrm{spec}_t^\gamma(e)$ that we will use to instantiate the construction with. Here, the γ parameter is a ghost name and t tracks whether the thread is a main thread $\mathcal{M}$ or a child thread $\mathcal{C}$. Additionally, we need a *specification context world* $\mathrm{CTX}^\gamma(\mathrm{W}, \Psi)$ that tracks the state of the concurrency handler.

$$\mathrm{spec}_t^\gamma(e) \triangleq \exists k, r.\, \boxed{\circ\{(k, r, t)\}}^\gamma * (\forall K.\, \mathrm{spec}(K[k\ r]) \mathrel{-\!\!*} \Rrightarrow_\bot \mathrm{spec}(K[e]))$$

$$\mathrm{CTX}^\gamma(\mathrm{W}, \Psi) \triangleq \exists B, \mathit{pool}.\, \mathsf{isBag}(B, \mathit{pool}) * \boxed{\bullet B}^\gamma * \mathrm{espec}_\mathrm{W}\ \mathtt{run}_{\mathtt{conc}}.\mathtt{go}\ \mathit{pool}\ \langle\Psi\rangle$$

In these definitions, we assert that there is some instance of the concurrency handler from Figure 4 installed, and we use ghost state to track the thread pool in the handler. Specifically, the thread pool is tracked using ghost resources such that $\boxed{\bullet B}^\gamma * \boxed{\circ B'}^\gamma \vdash B' \subseteq B$ and $\boxed{\bullet B}^\gamma \vdash \Rrightarrow_\mathrm{W} \boxed{\bullet (B \uplus B')}^\gamma * \boxed{\circ B'}^\gamma$. In the definition of $\mathrm{spec}_t^\gamma(e)$ we use this ghost state to assert that a triple of the form (k, r, t) is stored in the pool, where k is the continuation representing the thread and r is the result of the last effect. Additionally, we require that there is some way to run $k\ r$ so that it will reach the expression e. In other words, the thread currently in the pool may not yet be e, but when it is next scheduled to run, it can execute to e. The $\mathrm{CTX}^\gamma(\mathrm{W}, \Psi)$ assertion enforces that in fact there is an underlying espec running the concurrency handler providing the protocol Ψ.

This thread-local specification resource fulfills the requirements needed to instantiate genspec, *i.e.,* when $e \to^* e'$ then $\mathrm{spec}_t^\gamma(e) \vdash \Rrightarrow_\bot \mathrm{spec}_t^\gamma(e')$. When updating $\mathrm{spec}_t^\gamma(e)$ to $\mathrm{spec}_t^\gamma(e')$ in this derivation, instead of directly updating the underlying base effect specification resource in $\mathrm{CTX}^\gamma(\mathrm{W}, \Psi)$, we instead accumulate the evidence that there exists a thread in the pool that can be evaluated to the expression e' when it is next scheduled. Here, because a thread takes at most one effectful step before the next scheduling, an update modality is sufficient to carry that evidence; later in §7, we will see another application of this technique that requires accumulating non-trivial evidence.

Let $\mathrm{espec}_\mathrm{W}^{\gamma;t}\ e\ \langle\Psi\rangle$ be the notation for the instantiation of genspec with the $\mathrm{spec}_t^\gamma(e)$ assertion. This thread-local effect specification resource gives us a unified mechanism to reason about both global and thread-local effects. It supports analogues of the Espec-Pure and Espec-Do rules. In addition, we derive a rule for forking threads,

Spec-Fork

$$\frac{\mathrm{espec}_\mathrm{W}^{\gamma;t}\ N[\mathtt{fork}\ e]\ \langle \mathrm{CONC}_s^\gamma(\Psi)\rangle \qquad \mathrm{CTX}^\gamma(\mathrm{W}', \Psi') \sqsubseteq \mathrm{W}}{\Rrightarrow_\mathrm{W} \mathrm{espec}_\mathrm{W}^{\gamma;t}\ N[()]\ \langle \mathrm{CONC}_s^\gamma(\Psi)\rangle * \mathrm{espec}_{\mathrm{CTX}^\gamma(\mathrm{W}',\Psi')}^{\gamma;\mathcal{C}}\ e\ \langle \mathrm{CONC}_s^\gamma(\Psi')\rangle}\,*$$

where $\mathrm{CONC}_s^\gamma(\Psi) \triangleq \mathrm{FORK}_s^\gamma \oplus \Psi$ and $\mathrm{FORK}_s^\gamma(v, \Phi) \triangleq \exists e.\, v = (\mathtt{fork}, \lambda_.\, e) * (\mathrm{spec}_C^\gamma(e) \mathrel{-\!\!*} \Phi(()))$. Note that Spec-Fork assumes that the specification context is in the current world. The specification context is allocated when the concurrency handler is installed using the following rule.

Spec-ConcRun

$$\frac{\mathrm{espec}_W\ \mathtt{run}_{\mathtt{conc}}\ \mathit{main}\ \langle\Psi\rangle \qquad \mathtt{fork} \notin \mathrm{tags}(\Psi)}{\Rightarrow\!\!\!\Rightarrow_W \exists\gamma.\, \mathrm{CTX}^\gamma(W, \Psi) * \mathrm{espec}^{\gamma;\mathcal{M}}_{\mathrm{CTX}^\gamma(W,\Psi)}\ \mathit{main}\ ()\ \langle \mathrm{CONC}_s^\gamma(\Psi)\rangle}\,*$$

4.4 Logical Relation for Contextual Equivalence

Using RelFicus, we next define a *logical relation* in the style of Dreyer et al. [23] for proving contextual equivalence of programs written in typed subsets of FicusLang. Two expressions e_1 and e_2 are contextually equivalent at type τ if no well-typed context C can distinguish them; that is, replacing e_1 with e_2 in any client program does not change its observable behavior. Because programs in FicusLang may raise effects, we define this relative to a closing handler context H that handles them. Contextual equivalence $e_1 \simeq^H_{\mathrm{ctx}} e_2 : \tau$ is the symmetric interior of *contextual refinement* $e_1 \lesssim^H_{\mathrm{ctx}} e_2 : \tau$, which says that the observable behaviors of $H[C[e_1]]$ are included in those of $H[C[e_2]]$ for every well-typed context C:

$$e_1 \lesssim^H_{\mathrm{ctx}} e_2 : \tau \quad \triangleq \quad \forall b \in \mathbb{B}, C : \tau \to \mathbf{bool}.\, H[C[e_1]] \to^* b \Rightarrow H[C[e_2]] \to^* b.$$

Since H encloses both C and e_i, the context and the programs may interact through effects.

As an example, we consider a standard System-F-style type system $\Theta \mid \Gamma \vdash e : \tau$ with impredicative polymorphism, recursive types, and typing rules for CAS, FAA, and the fork operation (see, *e.g.,* Timany et al. [57] for a complete definition).

The logical relation is entirely standard and follows previous Iris-based models [57], *except* that we define the expression interpretation using RelFicus instantiated with the atomic heap instructions and concurrency. The expression interpretation is

$$[\![\tau]\!]^E(e_1, e_2) \triangleq \forall N, t.\, \mathrm{espec}^{\gamma;t}_{W_s}\ N[e_2]\ \langle\Psi_2\rangle \mathrel{-\!\!*} \mathrm{ewp}_W\ e_1\ \langle\Psi_1\rangle\ \{v_1.\, \exists v_2.\, \mathrm{espec}^{\gamma;t}_{W_s}\ N[v_2]\ \langle\Psi_2\rangle * [\![\tau]\!]^V(v_1, v_2)\}$$

where $\Psi_2 \triangleq \mathrm{CONC}_s^\gamma(\Psi')$, $\Psi_1 \triangleq \mathrm{CONC}_W(\Psi)$, $W_s = \mathrm{CTX}^\gamma(\bot, \Psi')$, $W = W_s \oplus \mathrm{TokI}(\top)$ for Ψ and Ψ' that describe the global stack of effects (state, heap, and atomic heap). The proof of the fundamental theorem of logical relations is immediate from the existing proofs since all our rules for reasoning about the atomic heap operations and concurrency are identical to the usual separation logic rules.

Theorem 4.2 (Fundamental). *If* $\Theta \mid \Gamma \vdash e : \tau$ *then* $\Theta \mid \Gamma \vDash e \lesssim e : \tau$.

To prove soundness, we consider the closing handler context

$$H_{\mathrm{CONC}} = \mathtt{run}_{\mathtt{state}}\ (\lambda_.\, \mathtt{run}_{\mathtt{heap}}\ (\lambda_.\, \mathtt{run}_{\mathtt{atomheap}}\ (\lambda_.\, \mathtt{run}_{\mathtt{conc}}\ (\lambda_.\, \bullet))))\ ()$$

and use the handler rules, *e.g.,* Ewp-ConcRun and Spec-ConcRun, and Theorem 4.1.

Theorem 4.3 (Soundness). *If* $\cdot \mid \cdot \vDash e_1 \lesssim e_2 : \tau$ *then* $e_1 \lesssim^{H_{\mathrm{CONC}}}_{\mathrm{ctx}} e_2 : \tau$.

Contextual Equivalence of Preemption. Using our logical relation, we prove that inserting additional preemption points in concurrent programs under the $\mathtt{run}_{\mathtt{conc}}$ scheduler does not change program behaviors. This justifies the soundness of using a scheduler that only triggers preemption when effects are raised, since it shows that additional preemption would not affect observable behavior. Formally, we introduce an expression **yield** that triggers a preemption point by defining **yield** $\triangleq$ **fork** (), *i.e.,* a program that simply forks a thread that terminates immediately. By forking a thread, **yield** transfers control to the scheduler, which may choose another thread to continue.

To justify that **yield** has no effect on the computation, we show that it is contextually equivalent to the unit value, *i.e.*, $\mathbf{yield} \simeq^{H_{\text{CONC}}}_{\text{ctx}} () : \mathbf{unit}$. The proof is an immediate consequence of the rules Ewp-Fork and Spec-Fork for the left-to-right and right-to-left refinements, respectively. As a corollary, for example, it then follows that $e_1;\ \mathbf{yield};\ e_2 \simeq^{H_{\text{CONC}}}_{\text{ctx}} e_1;\ e_2 : \tau$ for any well-typed e_1 and e_2. Similar arguments justify our crash model in §6 and stronger atomicity rules for distributed execution in §7.

Like many results for concurrent relational logics, this equivalence relies on $\mathtt{run}_{\mathtt{conc}}$ being fully non-deterministic: it may pick any thread to run next, and other threads may continue running after the main thread terminates. We argue that this suffices for safety properties. An interleaving produced by any other scheduler, including one that preempts between pure steps, can be reproduced under $\mathtt{run}_{\mathtt{conc}}$ by inserting **yield**s, which do not change the behavior. Hence, proving an ewp under the ATOM protocol implies that the program is safe under any scheduler.

5 Case Study: Prophecy Variables

When verifying certain concurrent programs in a forward-reasoning style, at some points in the proof it is necessary to know how later operations will be non-deterministically ordered. Prophecy variables [1, 32] are a logical mechanism that allows for "speculating" or "predicting" these future outcomes during a proof. In Iris, these prophecy variables are ghost code, and a prover must instrument a program to attach prophecy variables to operations whose values need to be predicted. New prophecy variables are allocated using a command **newproph**, and then attached to a program value using **resolve**. The proof rules for prophecy variables tell us *at the time of allocation* what the future resolved value will be. Iris comes with an additional proof showing that these prophecy variable operations can be erased from the program without affecting the outcome.

In this section, we show how to extend Ficus with prophecy variables. Our starting point is a single *global* prophecy variable. On top of this global prophecy variable, we implement effect handlers that allow for dynamically allocatable local prophecy variables, with an interface similar to that of Iris. Finally, we observe that by instrumenting the *handlers* for heap operations with prophecy variables, we can automatically extend all heap operations to have prophecies, without requiring the *client* program to be directly instrumented with prophecies.

Global Prophecy. Recall that FicusLang has a ghost expression **observe** v, which records the value v on a global trace. The entire future value of this trace is predicted in a global prophecy assertion $\mathsf{primProph}(\vec{v})$, which is used when performing an **observe**.

$$\textsc{Ewp-Obs}\quad \frac{\mathsf{primProph}(\vec{v})}{\mathsf{ewp}_{\mathrm{W}}\ \mathbf{observe}\ w\ \langle\Psi\rangle\ \{_.\ \exists \vec{v}'.\ \vec{v} = w :: \vec{v}' * \mathsf{primProph}(\vec{v}')\}}\ *$$

By making an observation of w, we immediately learn that the first element of $\vec{v}$ is indeed w, so $\vec{v}$ must equal $w :: \vec{v}'$ for some unknown $\vec{v}'$.[5] The adequacy theorem of Ficus is extended to provide this prophecy assertion.

Theorem 5.1 (Adequacy, Ficus). *Let φ be a first-order predicate. If* $\mathsf{primProph}(\vec{v}) \vdash \mathsf{ewp}\ e\ \langle\bot\rangle\ \{\varphi\}$ *for all $\vec{v}$, then executing e never gets stuck, and if $e \to^* v$ then $\varphi(v)$ holds.*

However, because this prophecy variable is global, it is awkward to use when trying to do local reasoning about data structures that need prophecies.

[5] As usual with prophecy reasoning, if the predicted value at the head of the sequence $\vec{v}$ was *not* w, then we derive a contradiction from this rule, and no longer have to reason about this moot execution with a misprediction.

Encoding Local Prophecy Variables. We recover Iris-style local prophecy variables by using handlers on top of the global `observe`. Formally, our local prophecy variables are specified by the following protocols:

$$\mathsf{NEWPROPH}(u, \Phi) \triangleq u = (\texttt{newproph}, ()) * (\forall p, \vec{v}.\, \mathsf{proph}(p, \vec{v}) \mathrel{-\!\!*} \Phi(p))$$
$$\mathsf{RESOLVE_PROPH}(u, \Phi) \triangleq \exists v, p, w, \vec{v}.\, u = (\texttt{resolve_proph}, (v, p, w)) * \mathsf{proph}(p, \vec{v}) * (\forall \vec{v}'.\, \vec{v} = (v, w) :: \vec{v}' * \mathsf{proph}(p, \vec{v}') \mathrel{-\!\!*} \Phi(v))$$

A new prophecy variable is created by raising the effect `newproph`, which returns a fresh prophecy variable p. Assertion $\mathsf{proph}(p, \vec{v})$ is the local version of $\mathsf{primProph}(\vec{v})$, which says that the trace of resolutions that will occur on prophecy variable p is $\vec{v}$. The effect `resolve_proph` resolves p to a pair of values (v, w). The first component v is the primary value that we want to observe, while the second component w is used to tag certain kinds of prophecies with metadata.

Under the hood, these assertions work by slicing the observation trace of the global prophecy into traces of individual prophecy variables. This is done by making every call to `observe` have the format (p, v, w), where p is the identifier of the prophecy variable that the observation belongs to. We can now try defining $\mathsf{proph}(p, \vec{v})$ as something like $\exists \vec{v}_0.\, \mathsf{primProph}(\vec{v}_0) * \vec{v} = \mathsf{filter}(p, \vec{v}_0)$, where filter is the least fixed point of

$$\mathsf{filter}(p, (p', v, w) :: \vec{v}) \triangleq \text{if } p = p' \text{ then } (v, w) :: \mathsf{filter}(p, \vec{v}) \text{ else } \mathsf{filter}(p, \vec{v})$$
$$\mathsf{filter}(p, _) \triangleq \epsilon$$

The function filter projects out the observations that are associated with the indicated prophecy variable, truncating the trace at the first observation that does not match the expected format.

Recall that resources in CSL are exclusive to one thread, so to actually have mutually independent prophecy variables, we need to decompose ownership of the global prophecy $\mathsf{primProph}(\vec{v})$ into ownership of individual prophecy variables using an authoritative ghost resource algebra.

$$\mathsf{proph}(p, \vec{v}) \triangleq \boxed{\circ\, [p \leftarrow \vec{v}]}^{\gamma_{\mathsf{proph}}}$$
$$I \triangleq \exists \vec{v}_0, M.\, \mathsf{primProph}(\vec{v}_0) * \boxed{\bullet M}^{\gamma_{\mathsf{proph}}} * \mathop{\Large *}_{p \mapsto \vec{v} \in M} \vec{v} = \mathsf{filter}(p, \vec{v}_0)$$

Here γ_{proph} is an arbitrary but globally fixed ghost name. Invariant I connects individual prophecy variables to the global prophecy and is maintained by the handler. The underlying resource algebra guarantees that the entry $p \mapsto \vec{v}$ is always in M.

The handler $\mathsf{run}_{\mathsf{proph}}$ providing protocols NEWPROPH and RESOLVE_PROPH is shown in §G.5. To create new prophecy variables, it internally maintains a monotonically increasing counter for the next fresh prophecy ID so that it can always "slice out" an unused resolving sequence from the global prophecy for a new prophecy variable. To resolve prophecy variable p to (v, w), it makes an observation of (p, v, w) and updates the ghost resources accordingly. In practice, $\mathsf{run}_{\mathsf{proph}}$ is installed as the outermost handler so that other handlers can use prophecy variables.

Atomic Prophecies. The resolve effect we have seen so far resolves a value that is returned by evaluating some expression. In other words, the prophecy is resolved *after* the expression finishes. However, in some scenarios, it is necessary to atomically execute the expression and prophecy resolution at the same time, particularly for atomic heap operations like CAS and FAA. Iris provides support for this so-called atomic prophecy resolution, and we can also implement this on top of the local prophecy variables through another effect handler with the following protocol:

$$\mathsf{RESOLVE}(\Psi)(u, \Phi) \triangleq \exists v, p, w, \vec{v}.\, u = (\texttt{resolve}, (v, p, w)) * \mathsf{proph}(p, \vec{v}) * \Psi(v, (\lambda r.\, \forall \vec{v}'.\, \vec{v} = (r, w) :: \vec{v}' \mathrel{-\!\!*} \mathsf{proph}(p, \vec{v}') \mathrel{-\!\!*} \Phi(r)))$$

```
run_crash_trigger ≜ λ main.
  if nondet_bool () then do (crash, ())
  else deep-try main () with
    v k ⇒ let r := do v in
          if nondet_bool () then do (crash, ())
          else k r
  | ret v ⇒ v
```

```
run_crash ≜ rec run main.
  deep-try main () with
    v k ⇒ match v with
          (crash, ()) ⇒ observe (); run main
        | (et, w) ⇒ k (do (et, w))
  | ret v ⇒ v
```

Note: the **deep-try** expression here installs a deep handler that is reinstalled after an effect is raised. It is implemented by using **try** and recursion.

Fig. 7. A model of crash-and-recovery execution.

Here, v is a pair of (tag, value) raised by a **do** expression. For example, to resolve a prophecy variable p to w along with a store expression $\ell \leftarrow x$, one needs to write **do** (resolve, ((store, (ℓ, x)), p, w)). The handler $\texttt{run}_{\texttt{atomproph}}$ providing the RESOLVE protocol is shown in § G.6. To handle **do** (resolve, (v, p, w)), it executes **do** (resolve_proph, (**do** v, p, w)). By installing $\texttt{run}_{\texttt{atomproph}}$ *inside* $\texttt{run}_{\texttt{proph}}$ and the handlers for heap operations, but *outside* $\texttt{run}_{\texttt{conc}}$ for concurrency, we ensure that the two **do**s, *i.e.,* **do** v and **do** (resolve_proph, ...), behave as if they executed simultaneously, because the additional **do** in $\texttt{run}_{\texttt{atomproph}}$ does not trigger an additional preemption.

Implicit Prophecies. As in HeapLang, the above interface for prophecies still requires a proof developer to annotate a program with calls to create new prophecies and to resolve them at relevant points. However, we can use effect handlers to give *every* heap location an associated prophecy variable that predicts the full trace of operations that will be performed on that heap location. This "prophetic heap" handler interposes on all heap-related effect tags, and adds an extra resolve operation before re-raising the effect. With this handler, when a location is allocated, in addition to the standard points-to assertion $\ell \mapsto v$, we also get a $\text{proph}(\ell, \vec{v})$ assertion. When a heap operation on ℓ occurs, we also pass this $\text{proph}(\ell, \vec{v})$ assertion, allowing us to deduce that the value read from or written to the location matches the head value in the trace $\vec{v}$. This allows for proofs with prophecies *without* having to annotate a client program with explicit prophecy operations. §B describes the protocols in further detail.

6 Case Study: Crash-Recovery Reasoning

In this section, we show how to model the effect of sudden interruption by crashes, like those caused by power loss. Many software systems that store data on durable media such as disks must be *crash safe*, meaning that the system must be able to recover from such a crash. When a crash occurs, any data that the system has in volatile memory, such as RAM, will be wiped, but data in durable storage will be preserved. After the system restarts, it will typically re-run a recovery procedure that restores system invariants. A number of program logics and verification frameworks have been developed for reasoning about such systems [13, 15, 43, 48]. After implementing effect handlers to encode crashes and recovery with durable state, we apply Ficus to derive protocols for reasoning about these systems in the style of Perennial [13], a separation logic for reasoning about the combination of concurrency and crash safety.

The process of crashing and recovering is modeled by the pair of handlers in Figure 7. The first handler $\texttt{run}_{\texttt{crash_trigger}}$ is installed at the inner-most level of a stack of effect handlers for each thread, allowing it to interpose on every **do**.[6] When an effect is raised, this handler first re-raises the effect to obtain its result, and then non-deterministically chooses to either trigger a crash by raising crash, or return the result to the continuation. Additionally, the handler also non-deterministically chooses to crash before *main* starts to execute. The second handler $\texttt{run}_{\texttt{crash}}$ responds to this trigger

[6]To install $\texttt{run}_{\texttt{crash_trigger}}$ for every child thread, our Rocq mechanization actually integrates $\texttt{run}_{\texttt{crash_trigger}}$ into $\texttt{run}_{\texttt{conc}}$.

by throwing away the captured continuation k and restarting the system by running *main*. Note that this handler does not directly deal with wiping the volatile state of the system. Instead, this is handled implicitly: by installing handlers for volatile state (such as the heap handler) *after* this crash handler (*i.e.,* as part of *main*), this volatile state will be effectively thrown away as a result of re-running *main* from scratch. In contrast, durable state can be preserved by installing these handlers *before* installing the crash handler at an outer level.

6.1 Managing the Crash Invariant

In order to establish that a system is crash safe, it is essential to show that when the system restarts, the *main* procedure finds itself in a state that satisfies its precondition. Perennial maintains a global *crash invariant* $\mathcal{R}$ that must hold before and after each step of execution, and which describes the durable state that the system needs upon restart.

Local Crash Conditions. Reasoning about a global crash invariant would run counter to the principle of local reasoning in concurrent separation logic. To recover per-thread reasoning about the crash invariant, Perennial extends the weakest precondition of each thread with an assertion called a *crash condition*. The crash condition enforces the portion of the global crash invariant that a given thread owns. In Ficus, rather than changing the weakest precondition to add an additional component, we can instead capture this local crash condition through worlds and a protocol transformer called DURA. We write $\mathsf{CC}(R)$ for a world stating a thread is responsible for ensuring that the local crash condition R holds after each step it takes.[7] The DURA protocol forces a thread to show that this R holds after each step of execution.

$$\mathsf{DURA}_W(\Psi)(v, \Phi) \triangleq \exists R.\, \Psi(v, \lambda r.\ {}_{\bot}\Rrightarrow_{W \oplus \mathsf{CC}(R)} (R \wedge {}_{W \oplus \mathsf{CC}(R)}\Rrightarrow_{\bot} \Phi(r)))$$

Note that R and the postcondition are connected by a logical conjunction $\wedge$ because the program can only either crash or continue, so only one of R and the postcondition will be used.

The derived proof rules for the crash handlers then require a specification for the top level *main* procedure of the following form.

$$\mathcal{R} \vdash \mathsf{ewp}_{W \oplus \mathsf{CC}(\blacklozenge \mathcal{R}), \bot}\ \mathit{main}\ ()\ \langle \mathsf{DURA}_W(\Psi) \rangle\ \{r.\ \exists R.\ {}_{\bot}\Rrightarrow_{W \oplus \mathsf{CC}(R)} R \wedge \Phi(r)\}$$

Here, $\mathcal{R}$ is the global crash invariant that also serves as the precondition for *main*, and $\blacklozenge$ is called the *post-crash modality* [54, 62], which captures how crashing modifies volatile and durable resources. Intuitively, $\blacklozenge\mathcal{R}$ says that $\mathcal{R}$ holds immediately after a crash. This top level specification says that the main thread starts with precondition $\mathcal{R}$, and the initial crash condition requires $\mathcal{R}$ holds after a crash. The main thread is free to alter the crash condition (by allocating/returning crash borrows using rules introduced later). But when raising an effect, as specified by the DURA protocol, the main thread must update to a world containing $\mathsf{CC}(R)$, which is the current crash condition, and prove R. When the main thread terminates, the top level specification also requires showing what the current crash condition is and proving that crash condition.

Concurrency with Crashes. To add support for concurrency, we install the concurrency handler inside the outer $\mathtt{run}_{\mathtt{crash}}$ handler. Because of the crash condition, we need to strengthen the CONC protocol to a protocol called CRASHCONC.

$$\begin{aligned}
&\mathsf{CRASHCONC}_W(\Psi) \triangleq \mathsf{DURA}_W(\Psi \oplus \mathsf{FORK}'_W(\Psi)) \\
&\quad \mathsf{FORK}'_W(\Psi)(v, \Phi) \triangleq \exists e.\, v = (\mathtt{fork}, \lambda_.\, e) * \\
&\qquad \triangleright\, \mathsf{ewp}_{W \oplus \mathsf{CC}(\mathsf{True}), \bot}\ e\ \langle \mathsf{CRASHCONC}_W(\Psi) \rangle\ \{_.\ \exists R_e.\ {}_{\bot}\Rrightarrow_{W \oplus \mathsf{CC}(R_e)} R_e\} * \Phi(())
\end{aligned}$$

[7] The notation CC stands for crash condition.

$$\begin{aligned}
\mathsf{ADISK} &\triangleq \mathsf{ADISK_LOAD} \oplus \mathsf{ADISK_STORE} \oplus \mathsf{ADISK_BARRIER} \\
\mathsf{ADISK_LOAD}(u,\Phi) &\triangleq \exists \ell, v_c, v.\, u = (\texttt{adisk_load}, \ell) * \ell \mapsto^{\mathrm{d}} [v_c]v * (\ell \mapsto^{\mathrm{d}} [v_c]v \mathrel{-\!\!*} \Phi(v)) \\
\mathsf{ADISK_STORE}(u,\Phi) &\triangleq \exists \ell, v_c, v, w.\, u = (\texttt{adisk_store}, (\ell, w)) * \ell \mapsto^{\mathrm{d}} [v_c]v * \\
&\qquad (\forall v_c' \in \{w, v_c\}.\, \ell \mapsto^{\mathrm{d}} [v_c']w \mathrel{-\!\!*} \Phi(())) \\
\mathsf{ADISK_BARRIER}(u,\Phi) &\triangleq \exists M.\, u = (\texttt{adisk_barrier}, ()) * \left(\textstyle\ast_{\ell \mapsto (v_c, v) \in M} \ell \mapsto^{\mathrm{d}} [v_c]v\right) * \\
&\qquad \left(\left(\textstyle\ast_{\ell \mapsto (v_c, v) \in M} v_c = v * \ell \mapsto^{\mathrm{d}} [v_c]v\right) \mathrel{-\!\!*} \Phi(())\right) \\
\textbf{adisk_load}\ \ell &\triangleq \textbf{do}\ (\texttt{adisk_load}, \ell) \quad \textbf{adisk_store}\ \ell\ v \triangleq \textbf{do}\ (\texttt{adisk_store}, (\ell, v)) \\
&\textbf{adisk_barrier} \triangleq \textbf{do}\ (\texttt{adisk_barrier}, ())
\end{aligned}$$

Fig. 8. The asynchronous disk protocol.

CRASHCONC follows the same structure as CONC and is also a guarded fixed point. However, a forked child thread starts with a trivial crash condition and can terminate with any crash condition.

When forking a child thread, one would naturally like to move some resources from the parent thread to the child thread. Similarly, synchronization primitives like locks are logically thought of as transferring ownership of resources in CSL. However, in order to transfer ownership of durable resources that might be part of the crash condition, we also need a mechanism to transfer the *obligation* to maintain that part of the crash condition. Crash borrows in Perennial [56] provide a mechanism to "borrow" part of the resources governed by the crash condition and transfer them to the child thread. A crash borrow $\boxed{P \mid R}$ has *content* P that describes the resources currently contained, and an associated *crash obligation* R, where $\Box(P \mathrel{-\!\!*} R)$.

The crash borrow can be understood as a box that packages up a resource P while preserving the obligation to ensure R holds in the event of a crash. Crash borrows are used through the following two key rules.

$$\frac{\triangleright P \qquad \Box(P \mathrel{-\!\!*} R)}{\mathrm{CC}(Q * R) \Rrightarrow_{\mathrm{CC}(Q)} \boxed{P \mid R}}\mathrel{-\!\!*} \qquad \text{Wupd-CBrwAlloc}$$

$$\frac{\boxed{P \mid R}}{\mathrm{CC}(Q) \Rrightarrow_{\mathrm{CC}(Q * R)} \triangleright P}\mathrel{-\!\!*} \qquad \text{Wupd-CBrwReturn}$$

Rule Wupd-CBrwAlloc creates a crash borrow $\boxed{P \mid R}$. It consumes a resource P that is stronger than R and removes R from the crash condition. Rule Wupd-CBrwReturn opens the box $\boxed{P \mid R}$ to extract resource P, and in exchange, it adds R to the crash condition. In Perennial, this crash borrow mechanism is encoded on top of standard Iris invariants in a complex manner that requires extensive use of later credits [51] to avoid inconsistencies from impredicative circularities. In Ficus, the encoding is considerably simpler, because we are able to use a separate world for managing crash conditions and crash borrows. The complete model can be found in §C.

6.2 Asynchrony and Crash-Aware Prophecies

Many durable storage media are *asynchronous*, meaning that when a write is performed, the written value does not immediately become durable. Instead, the written value is first stored in some volatile buffer. These buffers are then non-deterministically flushed to disk, making the writes durable. Alternatively, a program can force a flush by issuing a *barrier*. If a crash occurs while the value is still in the volatile buffer, then the write is lost. Reasoning about asynchrony is challenging when trying to prove that a concurrent durable data structure satisfies *durable linearizability* [30], because it makes the durability of an operation *future dependent*.

To deal with this challenge, Perennial introduced a *prophetic disk points-to* assertion of the form $\ell \mapsto^{\mathrm{d}} [v_c]v$ which says that the disk address ℓ currently stores the value v, and after a crash occurs, the stored value will be v_c. This property is captured in the following entailment that describes how the disk points-to changes under the post-crash modality: $\ell \mapsto^{\mathrm{d}} [v_c]v \vdash \blacklozenge \ell \mapsto^{\mathrm{d}} [v_c]v_c$. Essentially, this assertion bundles a normal points-to with a form of prophecy about the post-crash state. However, in Perennial, this primitive could not re-use the existing support for prophecies in Iris, and instead has an ad-hoc soundness proof. The issue is that, with standard Iris prophecy variables, there is no way to make a prophecy about whether an event will happen before or after a crash.

In contrast, in Ficus it is easy to handle this by incorporating prophecy resolution as part of the implementation of the crash handler. We use the **observe** () statement in $\mathtt{run}_{\mathtt{crash}}$ to effectively record that a crash has occurred in the trace of every prophecy variable. Formally, for every prophecy variable p, we have $\mathsf{proph}(p, \vec{v}) \vdash \blacklozenge(\vec{v} = \epsilon)$. The definition of filter in §5 ensures that this truncates the trace of events in the prophecy stream for all variables. Thus, when inspecting the prophecy stream, it is possible to determine whether a crash will occur before the prophecy is resolved. We call these resulting prophecy variables *crash-aware*. Using this mechanism, we implement an asynchronous disk with prophetic points-to assertions by allocating a crash-aware prophecy for each buffered write, which predicts whether the write will be flushed before a crash. The protocol of the asynchronous disk is found in Figure 8 and more details can be found in §C. ADISK_LOAD returns the current value v of a location. ADISK_STORE sets the current value to w, and the prophecy allocated for this write determines the post-crash value: the client must handle both $v_c' = w$, where the write is flushed before a crash, and $v_c' = v_c$, where the write is lost and the location keeps its previous post-crash value. Finally, ADISK_BARRIER takes ownership of a collection M of disk points-to assertions and returns them together with the fact that $v_c = v$ for each of them. Because a barrier flushes all pending writes and resolves their prophecies accordingly, the current value of each location is also the value it will have after a crash, so any prophesied future in which a write before the barrier is lost leads to a contradiction.

6.3 Recovering the Perennial Logic

The effects introduced above provide all of the features Perennial has in order to support crash reasoning, except that our crash model diverges from the standard crash model considered by prior work like Perennial: just as our concurrency handler only allowed preemption when effects were raised, our crash handler only allows crashing after each effectful step. In contrast, the typical model allows crashing between every step. However, just as with concurrency, this crash model is equivalent to the standard model for purposes of partial correctness: the pure steps do not affect the observable state of the system after a crash, so choosing to crash before or after a sequence of pure steps is equivalent to crashing at some point in the middle of them. To justify the soundness of our model, we again show that **yield** is contextually equivalent to the unit value under our crash model, similar to what we do in §4.4. This time, because a potential crash could happen after any effectful step, a **yield** serves as both a possible preemption point and a possible crash point.

Formally, we define a handler context $H_{\mathsf{PERENNIAL}}(d)$ that handles the effects for asynchronous disk, crash, heap, and concurrency. Concretely, it first sets the initial durable state (*i.e.,* the state of the disk) to d, then installs the $\mathtt{run}_{\mathtt{crash}}$ handler, and finally installs other handlers for volatile state inside. We then prove $\forall d.\ \mathbf{yield} \simeq_{\mathrm{ctx}}^{H_{\mathsf{PERENNIAL}}(d)} () : \mathbf{unit}$. The proof is structured by dividing up execution into a sequence of *rounds* separated by crashes. The proof reuses much of the construction of the spec resource in §4.3 and the logical relation in §4.4 to reason about the behavior of two programs within one round of execution. The major difference is that to show e_1 refines e_2 within one round, because e_1 could crash after any effect, the spec resource of e_2 must be able to crash

```
read_pair ≜ λ().
  let active := (if adisk_load ℓ_d = 1 then ℓ_1 else ℓ_2) in
  let a_0 := adisk_load active in
  let a_1 := adisk_load (active + 1) in
  (a_0, a_1)
```

```
write_pair ≜ λ(b_0, b_1).
1  (if adisk_load ℓ_d = 1 then
2    adisk_store ℓ_2 b_0;
3    adisk_store (ℓ_2 + 1) b_1;
4    adisk_barrier;
5    adisk_store ℓ_d 2
6  else ... (* symmetrically *));
7  adisk_barrier
```

$$\textsc{Ewp-ReadPair}\quad \frac{\mathsf{pair}(a_0,a_1) \qquad (\mathsf{pair}(a_0,a_1) \mathrel{-\!\!*} \Phi((a_0,a_1))) \wedge (\blacklozenge \mathsf{pair}(a_0,a_1) \mathrel{-\!\!*} R)}{\mathsf{ewp}_{\mathsf{TokI}(\top)\oplus\mathsf{CC}(R)}\ \texttt{read_pair}\ ()\ \langle \Psi_{pair} \rangle\ \{\Phi\}}*$$

$$\textsc{Ewp-WritePair}\quad \frac{\mathsf{pair}(a_0,a_1) \qquad (\mathsf{pair}(b_0,b_1) \mathrel{-\!\!*} \Phi(())) \wedge (\blacklozenge(\mathsf{pair}(a_0,a_1) \vee \mathsf{pair}(b_0,b_1)) \mathrel{-\!\!*} R)}{\mathsf{ewp}_{\mathsf{TokI}(\top)\oplus\mathsf{CC}(R)}\ \texttt{write_pair}\ (b_0,b_1)\ \langle \Psi_{pair} \rangle\ \{\Phi\}}*$$

$$\mathsf{pair}(a_0,a_1) \triangleq \ell_d \mapsto^{\mathrm{d}} [1]1 * \left(\ell_1 \mapsto^{\mathrm{d}} [a_0]a_0 * \ell_1 + 1 \mapsto^{\mathrm{d}} [a_1]a_1 * \ell_2 \mapsto^{\mathrm{d}} [_]_ * \ell_2 + 1 \mapsto^{\mathrm{d}} [_]_\right)$$

$$\vee\ \ell_d \mapsto^{\mathrm{d}} [2]2 * \left(\ell_2 \mapsto^{\mathrm{d}} [a_0]a_0 * \ell_2 + 1 \mapsto^{\mathrm{d}} [a_1]a_1 * \ell_1 \mapsto^{\mathrm{d}} [_]_ * \ell_1 + 1 \mapsto^{\mathrm{d}} [_]_\right)$$

$$\Psi_{pair} \triangleq \mathsf{DURA}_{\mathsf{TokI}(\top)}(\mathsf{ATOMHEAP} \oplus \mathsf{HEAP} \oplus \mathsf{ADISK})$$

Fig. 9. Implementation and a simplified specification of the durable pair.

angelically at any step as well. To derive a multi-round contextual refinement on top of that, we do an induction on the number of rounds and use the logical relation to relate the behavior of two programs in each round.

6.4 Example: the Durable Pair

To exercise the features introduced in this section, we verify a durable pair example that uses a crash-safety pattern called a *shadow copy*, which is used in many crash-safe systems.

Implementation. The implementation and specification of the durable pair are shown in Figure 9. Logically, each durable pair stores a pair of values that span two consecutive disk addresses. The goal is to be able to update the pair in a way that is atomic with respect to crashes. Naively writing one component and then writing the next does not work, because a crash in between the two writes, or a crash after one write's buffer has been flushed but not the other's, can lead to a state after recovery where only one of the components of the pair has been updated. In order to be able to update the pair in a crash-safe way, the implementation instead maintains two copies of the pair (at $(\ell_1, \ell_1 + 1)$ and $(\ell_2, \ell_2 + 1)$, respectively): one that is *active* and one that is *inactive*. The current active pair is recorded in a selector node ℓ_d. To read from the durable pair, `read_pair` reads from the selector node to learn which pair is active and then reads from the active one. To update the values, `write_pair` first does two writes to update the inactive copy. It then issues a barrier to ensure both of these writes are durable. Next, it does one write to the selector node to make this copy the active one, and issues another barrier to make this update durable. Our Rocq mechanization builds a thread-safe durable pair on top of the durable pair introduced here by wrapping it with a lock. For simplicity, here we focus on the sequential version, and just reason about the crash safety.

Specification. The assertion $\mathsf{pair}(a_0, a_1)$ says that the value of the pair on disk is (a_0, a_1): if the selector node ℓ_d equals 1, then locations ℓ_1 and $\ell_1 + 1$ hold the pair, while the values for ℓ_2 and

$\ell_2 + 1$ are unspecified, and vice versa. The specification Ewp-ReadPair for `read_pair` has two premises. The first premise is the precondition: we must own $\mathsf{pair}(a_0, a_1)$. The second premise gives the postcondition in a continuation passing style: to prove the postcondition Φ, we can assume ownership of $\mathsf{pair}(a_0, a_1)$, and in addition, we must show the crash condition R, getting access to $\blacklozenge\mathsf{pair}(a_0, a_1)$, *i.e.,* the pair still holds (a_0, a_1) after a crash. Rule Ewp-WritePair follows the same pattern, except that if a crash occurs during the write, we only learn that after the crash the pair holds either the old values (a_0, a_1) or the new values (b_0, b_1). In other words, the write is atomic with respect to crashes.

Verification. We sketch the proof of Ewp-WritePair; the proof of Ewp-ReadPair is simpler. Each disk operation in `write_pair` is an effect, which we step over using Ewp-Bind and Ewp-DoWupd. Because Ψ_{pair} is built with DURA, each such step comes with the obligation to show that the crash condition R holds afterwards. By the second premise of Ewp-WritePair, to show R, it suffices to show $\blacklozenge(\mathsf{pair}(a_0, a_1) \vee \mathsf{pair}(b_0, b_1))$.

We start by case splitting on the disjunction in $\mathsf{pair}(a_0, a_1)$. We will consider the left case here, where the selector ℓ_d indicates that the first copy of the pair is active; the other case is symmetric. Line 1 reads $\ell_d \mapsto^{\mathrm{d}} [1]1$ and returns 1. Lines 2 and 3 write (b_0, b_1) to the inactive copy, and the barrier on line 4 makes these writes durable, giving us $\ell_2 \mapsto^{\mathrm{d}} [b_0]b_0$ and $\ell_2 + 1 \mapsto^{\mathrm{d}} [b_1]b_1$. Because $\mathsf{pair}(a_0, a_1)$ says nothing about the inactive copy, it still holds after each of these steps, and $\mathsf{pair}(a_0, a_1) \vdash \blacklozenge\mathsf{pair}(a_0, a_1)$ discharges the crash condition.

The key step is the store to the selector ℓ_d on line 5. By the ADISK_STORE protocol, we obtain $\ell_d \mapsto^{\mathrm{d}} [v_c]2$ for some $v_c \in \{1, 2\}$, where the prophesied v_c tells us in advance whether this store will survive a crash. We case split on v_c: if $v_c = 2$, the store will be durable, so we now own $\mathsf{pair}(b_0, b_1)$, which gives the crash condition through the right disjunct; if $v_c = 1$, a crash would undo the store, so we have $\blacklozenge\mathsf{pair}(a_0, a_1)$, the left disjunct. Finally, the barrier on line 7 takes $\ell_d \mapsto^{\mathrm{d}} [v_c]2$ and gives it back with the fact $v_c = 2$, since the pending store to ℓ_d is now flushed. In the second case this contradicts $v_c = 1$, so only the first case remains, where we own $\mathsf{pair}(b_0, b_1)$ and conclude $\Phi(())$ using the second premise.

7 Case Study: Distributed Systems with IronFleet-Style Atomic Blocks

In this section, we consider a distributed system with multiple nodes connected by an unreliable network, in which nodes communicate through messages that may be dropped, delayed, duplicated, or re-ordered. On top of the network, a global scheduler decides the order of execution of nodes.

Network. The network provides two operations: $\mathsf{NETWORK} \triangleq \mathsf{SEND} \oplus \mathsf{RECV}$, where

$$\mathsf{SEND}(v, \Phi) \triangleq \exists s, t, m, M.\, v = (\mathsf{send}, (s, t, m)) * t \rightsquigarrow M * (t \rightsquigarrow \{(s, t, m)\} \cup M \mathrel{-\!\!*} \Phi(()))$$

$$\mathsf{RECV}(v, \Phi) \triangleq \exists t, M.\, v = (\mathsf{recv}, t) * t \rightsquigarrow M * \left(\begin{array}{c} \forall x.\, t \rightsquigarrow M * \big(x = \mathbf{inl}\ () \vee \\ (\exists s, m.\, x = \mathbf{inr}(s, t, m) \wedge (s, t, m) \in M)\big) \mathrel{-\!\!*} \Phi(x) \end{array} \right)$$

Assertion $t \rightsquigarrow M$ says that M is the set of messages that have ever been sent to address t. Since messages can be arbitrarily duplicated, this set is monotonically increasing *w.r.t.* the subset relation. The SEND protocol expects a package of (source address, destination address, message) as input, and adds this package to the message history of the destination address. The RECV protocol expects the destination address t as input and non-deterministically chooses to either not return a message or to return an arbitrary message that has been sent to t. The dropping of a message is implicitly modeled as just never having it be selected for receipt. The handler providing protocol NETWORK is called $\mathtt{run}_{\mathtt{network}}$, which implements the network as a soup of messages [34, 66]. The code for this handler can be found in §G.7.

Scheduler with IronFleet-Style Atomic Blocks. Next, we need a scheduler $\texttt{run}_{\texttt{distr}}$ (code shown in §G.8) that specifies how nodes run concurrently through a DISTR^t_W protocol.

$$\text{DISTR}^t_W \triangleq \text{ATOM}_W(\text{SEND} \oplus \text{START}^t_W) \oplus \text{RECV}$$

$$\text{START}^t_W(v, \Phi) \triangleq \exists e.\, v = (\texttt{start}, \lambda_.\, e) * \left(t = C \vee \triangleright\, \text{ewp}_W\, e\, \langle \text{DISTR}^C_W \rangle\, \{_.\, \text{True}\}\right) * \Phi(())$$

The $\texttt{run}_{\texttt{distr}}$ handler resembles $\texttt{run}_{\texttt{conc}}$ and potentially transfers control to different nodes when an effect is raised by a node. The START protocol is used for initially creating nodes. In the protocol above, the RECV effect is *outside* the ATOM protocol transformer. The reason for this is that the $\texttt{run}_{\texttt{distr}}$ scheduler does *not* transfer control to another node when processing a `recv` operation. In other words, a node can receive a series of messages without transferring control to another node. This modeling choice is inspired by IronFleet [27] which uses a movers-based parallel reduction proof [36] to treat a sequence of receives followed by a sequence of sends as an atomic step. Our global scheduler permits the prover to view a series of `recv` operations, followed by a series of node-local processing operations, followed by one `send` operation as an atomic block.[8] As a result, because the RECV is not included in the ATOM component, we do *not* need to close shared invariants when performing a RECV operation.

Even though this scheduler does not include preemptions at RECV, the absence of these preemptions does not affect the overall set of possible behaviors of the program. To prove this, we use RelFicus to prove that an explicit **`yield`** preemption is contextually equivalent to the unit value. Thus, adding in additional preemptions does not change the program's behavior. To carry out this proof, we develop a node-local specification resource $\text{specn}^\gamma_t(e)$, similar to the thread-local version described in §4.3. It also uses the evidence accumulation technique, but additionally accumulates the evidence that a node can delay message receipt without changing behavior. The idea is that if a message m is received at some time T, then if we delay that receipt to some later time T', it is still possible to receive m. We capture this evidence using a *monotonic* resource algebra to track the set M of messages. We then build a logical relation on top of the $\text{specn}^\gamma_t(e)$ resource in the style of §4.4 to complete the proof. More details can be found in §D.

8 Related Work

8.1 Program Logics for Effect Handlers

The most closely related work is the Hazel logic for effect handlers [19]. As discussed in §3, Ficus extends Hazel with support for extensible worlds. While Ficus restricts protocols to be monotonic, Hazel does not directly impose this requirement, but instead computes an upward closure of the protocol to enforce monotonicity. Both approaches are identical in practice because protocols defined in a "standard" way are already monotonic: protocols directly capturing the input condition and output condition (like the READ and WRITE protocols) are monotonic, and monotonicity is closed under the $\oplus$ combinator and all protocol transformers we have considered.

In both Ficus and Hazel, continuations are *one-shot*, meaning that an effect can only invoke the continuation once. Subsequent to Hazel, de Vilhena [18] developed Maze, a separation logic that supports multi-shot continuations. The lack of multi-shot continuations in Ficus means we could not implement backtracking nondeterminism. However, to support multi-shot continuations using Maze's approach, we would have to discard all non-persistent resources (*e.g.*, points-to) when raising a multi-shot effect, and it is not possible to compose this type of reasoning about multi-shot effects with our other effects. Introducing multi-shot continuations also breaks the frame rule (Ewp-Frame) because an expression could return multiple times.

[8] We preempt after a single send because a node could diverge after sending. IronFleet avoids this by proving total correctness.

Hazel only handles unary reasoning, whereas RelFicus supports relational reasoning through an encoding into Ficus. The Blaze logic developed by de Vilhena et al. [21] also supports relational reasoning for effect handlers. Like RelFicus, Blaze builds on a unary logic and represents a specification program via ghost state. However, unlike RelFicus, in which the unary logic and the specification program have separate protocols, Blaze instead provides a judgment with a *relational* protocol. They use this to prove refinements in which the interpretation of effects is different between the two programs. In contrast, our examples keep effects the same on both sides and prove that client programs under these effects are equivalent.

Among other examples, de Vilhena et al. [21] use Blaze to prove that a handler implementation of concurrency refines a primitive concurrency effect. This refinement is in some sense the opposite of the direction that motivated our refinement proof in §4: it essentially shows that for every execution of the concurrency handler (which only preempts at effects), there is a corresponding execution using primitive concurrency (which preempts at every step). In contrast, we show that inserting additional preemption points when using the concurrency handler does not generate new behaviors. This is morally equivalent to showing that the concurrency handler already covers all possible behaviors that could be generated by a full interleaving semantics. It would be interesting to apply Blaze's approach to the kinds of applications we have considered here to justify the soundness of alternate implementations of effects that allow for deriving stronger reasoning rules.

Like the language of Hazel, FicusLang does not natively support effect tags, but instead requires values of effects to have the form (tag,value) and expects the handlers to dispatch and re-raise effects based on the tag, whereas many other works on effect handlers [9, 10, 20, 21, 29, 47, 53, 61] bake effect tags into the syntax and operational semantics. The language-level tags make it possible to syntactically decide the set of tags handled by an evaluation context. Blaze [21] exploits this for a more general bind rule not limited to (absolutely) neutral contexts, but only requiring that the context does not handle tags of effects that may be raised by the expression being bound. Ficus does not use syntactic tags because restricting to syntactic tags prevents handlers like $\texttt{run}_{\texttt{conc}}$ from interposing on all effects that propagate to outer handlers.

Our logical relations consider type systems with a fixed collection of effects and do not provide rules for typing general effect handlers. Tes [20] and Affect [61] use logics to construct unary logical-relations models for type systems for effect handlers. Biernacki et al. [9, 10] directly construct a binary logical-relations model for effects and handlers using biorthogonality and step indexing.

8.2 Extensible Program Logics

As described in the introduction, Vistrup et al. [64] develop an approach to extensible program logics using ITrees [67]. They use a mechanism called *logical effect handlers* to interpret ITree events for an effect, which has similarities to the way Hazel and Ficus's protocols give a logical interpretation of what a raised effect will do. Their soundness proofs relate these logical effect handlers to interpretations of the effects. Similarly, Frumin et al. [24] develop *guarded* ITrees as a compositional model of higher-order programming languages with an extensible program logic. Neither Vistrup et al. [64] nor Frumin et al. [24] develop a relational logic. On the other hand, Vistrup et al. show how to model other features, such as total correctness and angelic non-determinism, which we do not consider. Initially, the ITree-based approach of Vistrup et al. [64] did not include support for prophecy variables. However, in subsequent work on the Mosaic framework [63], Vistrup et al. showed how to extend the ITree-based approach to also handle prophecies. A further benefit of Mosaic is that it allows composing extensions to the *pure* syntax of the language, whereas Ficus's pure calculus is fixed.

Like Ficus's extensible worlds, Matsushita and Tsukada [39] parameterize the Iris update modality and Hoare triples by a notion of a world. However, they require that the update shifts to the same

world before and after, *i.e.,* only shifts of the form ${}_{W}\Rightarrow_{W}$. Hence, they cannot model the use of extensible worlds in §3, in which invariants are kept open across non-preempting steps.

Dijkstra Monads [52] offer a framework for deriving pre- and postcondition reasoning about dependently-typed programs with monadic effects, and, more recently, some aspects of algebraic effect handlers [38]. One benefit of Dijkstra Monads is that they work well for representing effectful programs using a monadic, shallow embedding inside of a dependently-typed language like Rocq or Lean. Gladshtein et al. [25] have recently used this approach to develop a framework for verifying programs in what they call a *multi-modal* way, which combines a variety of automated and interactive verification techniques in a powerful, common framework. In particular, they note that their shallow embedding leads to a naturally executable semantics, which they contrast both with standard approaches to building program logics and with the work of Vistrup et al. [64]. While we have not explored this, one advantage of Ficus is that it in principle also leads to an executable semantics: one simply needs to write an interpreter for FicusLang, and then all implementations of effects become executable as well. It would be interesting as future work to see what other aspects of their multi-modal verification framework can be done using effect handlers.

While Gladshtein et al. and other prior works demonstrate some powerful benefits of Dijkstra Monads, certain effects and verification tasks are challenging to represent in a shallowly-embedded monadic way as compared to our deep embedding with effect handlers. For example, existing applications of Dijkstra Monads do not model fine-grained concurrency or crashes, and these may be challenging to encode as monads in a compositional way. And, some applications of program logics are most naturally carried out with explicit syntax: for example, building logical-relations models of type systems, where one needs to refer to the typing rules of the object language. Finally, existing frameworks based on Dijkstra Monads do not support relational reasoning.

9 Conclusion

We present Ficus, an extensible program logic that allows users to model new effects by writing programs in the form of effect handlers. We also present RelFicus, a relational logic that we have used to justify the soundness of non-standard encodings of various effects. As case studies, we use Ficus to recover Iris's HeapLang logic and crash-safety reasoning from Perennial, and construct a new separation logic for distributed systems in the style of IronFleet.

The key idea underlying Ficus is that effect handlers are just regular programs and therefore extending a program logic does not require skills distinct from those needed to use a program logic. But this does come with a trade-off: the executable nature of effect handlers prevents us from modeling certain language features that cannot be expressed as programs, like angelic choice, non-computable real numbers, and axiomatic weak memory models.

Another limitation is that Ficus only covers partial correctness. For the core Ficus logic, we do not foresee any fundamental obstacle to supporting total correctness. However, the contextual equivalence between our non-standard scheduler and a fully preemptive scheduler does not hold when termination is observable, so we would need to directly model a handler that can interrupt pure steps, which would require extending the effect handler logic to support asynchronous interruptions.

Finally, Ficus is only extensible by adding new effects; its core pure calculus is fixed. For future work, it would be interesting to find a way to combine the effect-handler approach to extensibility with a mechanism for extending the pure syntax of the language.

Data Availability Statement

The Rocq mechanization accompanying this work is available on Zenodo [68] and on GitHub at https://github.com/u8cat/ficus/tree/popl2027-aec-submission.

Acknowledgments

We thank François Pottier and the anonymous reviewers for their constructive feedback, which helped improve the presentation in the paper. This work was supported in part by the National Science Foundation under Grant No. 2319168 and Grant No. 2524669, as well as the Carlsberg Foundation under Grant No. CF23-0791. Any opinions, findings, and conclusions or recommendations expressed in this material are those of the authors and do not necessarily reflect the views of these funding agencies.

References

[1] Martín Abadi and Leslie Lamport. 1988. The Existence of Refinement Mappings. In *Proceedings of the Third Annual Symposium on Logic in Computer Science (LICS '88), Edinburgh, Scotland, UK, July 5-8, 1988*. 165–175. doi:10.1109/LICS.1988.5115

[2] Alejandro Aguirre, Gilles Barthe, Marco Gaboardi, Deepak Garg, Shin-ya Katsumata, and Tetsuya Sato. 2021. Higher-Order Probabilistic Adversarial Computations: Categorical Semantics and Program Logics. *Proc. ACM Program. Lang.* 5, ICFP (2021), 1–30. doi:10.1145/3473598

[3] Alejandro Aguirre, Philipp G. Haselwarter, Markus de Medeiros, Kwing Hei Li, Simon Oddershede Gregersen, Joseph Tassarotti, and Lars Birkedal. 2024. Error Credits: Resourceful Reasoning about Error Bounds for Higher-Order Probabilistic Programs. *Proc. ACM Program. Lang.* 8, ICFP (2024), 284–316. doi:10.1145/3674635

[4] Andrew W. Appel, Paul-André Melliès, Christopher D. Richards, and Jérôme Vouillon. 2007. A Very Modal Model of a Modern, Major, General Type System. In *Proceedings of the 34th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages, POPL 2007, Nice, France, January 17-19, 2007*. 109–122. doi:10.1145/1190216.1190235

[5] Jialu Bao, Emanuele D'Osualdo, and Azadeh Farzan. 2025. Bluebell: An Alliance of Relational Lifting and Independence for Probabilistic Reasoning. *Proc. ACM Program. Lang.* 9, POPL (2025), 1719–1749. doi:10.1145/3704894

[6] Gilles Barthe, Thomas Espitau, Benjamin Grégoire, Justin Hsu, Léo Stefanesco, and Pierre-Yves Strub. 2015. Relational Reasoning via Probabilistic Coupling. In *Logic for Programming, Artificial Intelligence, and Reasoning - 20th International Conference, LPAR-20 2015, Suva, Fiji, November 24-28, 2015, Proceedings*. 387–401. doi:10.1007/978-3-662-48899-7_27

[7] Gilles Barthe, Justin Hsu, and Kevin Liao. 2020. A Probabilistic Separation Logic. *Proc. ACM Program. Lang.* 4, POPL (2020), 55:1–55:30. doi:10.1145/3371123

[8] Kevin Batz, Benjamin Lucien Kaminski, Joost-Pieter Katoen, Christoph Matheja, and Thomas Noll. 2019. Quantitative Separation Logic: A Logic for Reasoning about Probabilistic Pointer Programs. *Proc. ACM Program. Lang.* 3, POPL (2019), 34:1–34:29. doi:10.1145/3290347

[9] Dariusz Biernacki, Maciej Piróg, Piotr Polesiuk, and Filip Sieczkowski. 2017. Handle with Care: Relational Interpretation of Algebraic Effects and Handlers. *Proc. ACM Program. Lang.* 2, POPL, Article 8 (Dec. 2017), 30 pages. doi:10.1145/3158096

[10] Dariusz Biernacki, Maciej Piróg, Piotr Polesiuk, and Filip Sieczkowski. 2020. Binders by Day, Labels by Night: Effect Instances via Lexically Scoped Handlers. *Proc. ACM Program. Lang.* 4, POPL (2020), 48:1–48:29. doi:10.1145/3371116

[11] Lars Birkedal, Rasmus Ejlers Møgelberg, Jan Schwinghammer, and Kristian Støvring. 2012. First Steps in Synthetic Guarded Domain Theory: Step-Indexing in the Topos of Trees. *Log. Methods Comput. Sci.* 8, 4 (2012). doi:10.2168/LMCS-8(4:1)2012

[12] John Boyland. 2003. Checking Interference with Fractional Permissions. In *Static Analysis*, Radhia Cousot (Ed.). Springer Berlin Heidelberg, Berlin, Heidelberg, 55–72. doi:10.1007/3-540-44898-5_4

[13] Tej Chajed, Joseph Tassarotti, M. Frans Kaashoek, and Nickolai Zeldovich. 2019. Verifying Concurrent, Crash-Safe Systems with Perennial. In *Proceedings of the 27th ACM Symposium on Operating Systems Principles, SOSP 2019, Huntsville, ON, Canada, October 27-30, 2019*. 243–258. doi:10.1145/3341301.3359632

[14] Tej Chajed, Joseph Tassarotti, Mark Theng, Ralf Jung, M. Frans Kaashoek, and Nickolai Zeldovich. 2021. GoJournal: A Verified, Concurrent, Crash-Safe Journaling System. In *15th USENIX Symposium on Operating Systems Design and Implementation (OSDI 21)*. USENIX Association, 423–439. https://www.usenix.org/conference/osdi21/presentation/chajed

[15] Haogang Chen, Daniel Ziegler, Tej Chajed, Adam Chlipala, M. Frans Kaashoek, and Nickolai Zeldovich. 2015. Using Crash Hoare Logic for Certifying the FSCQ File System. In *Proceedings of the 25th Symposium on Operating Systems Principles, SOSP 2015, Monterey, CA, USA, October 4-7, 2015*. 18–37. doi:10.1145/2815400.2815402

[16] Krzysztof Ciesielski. 2007. On Stefan Banach and Some of His Results. *Banach Journal of Mathematical Analysis* 1, 1 (2007), 1–10. doi:10.15352/bjma/1240321550

[17] Hoang-Hai Dang, Jaehwang Jung, Jaemin Choi, Duc-Than Nguyen, William Mansky, Jeehoon Kang, and Derek Dreyer. 2022. Compass: Strong and Compositional Library Specifications in Relaxed Memory Separation Logic. In *Proceedings of*

the 43rd ACM SIGPLAN International Conference on Programming Language Design and Implementation (San Diego, CA, USA) *(PLDI 2022)*. Association for Computing Machinery, New York, NY, USA, 792–808. doi:10.1145/3519939.3523451

[18] Paulo Emílio de Vilhena. 2022. *Proof of Programs with Effect Handlers*. Ph. D. Dissertation. Paris Cité University, France. https://tel.archives-ouvertes.fr/tel-03891381

[19] Paulo Emílio de Vilhena and François Pottier. 2021. A Separation Logic for Effect Handlers. *Proc. ACM Program. Lang.* 5, POPL, Article 33 (Jan. 2021), 28 pages. doi:10.1145/3434314

[20] Paulo Emílio de Vilhena and François Pottier. 2023. A Type System for Effect Handlers and Dynamic Labels. In *Programming Languages and Systems - 32nd European Symposium on Programming, ESOP 2023, Held as Part of the European Joint Conferences on Theory and Practice of Software, ETAPS 2023, Paris, France, April 22-27, 2023, Proceedings*. 225–252. doi:10.1007/978-3-031-30044-8_9

[21] Paulo Emílio de Vilhena, Simcha van Collem, Ines Wright, and Robbert Krebbers. 2026. A Relational Separation Logic for Effect Handlers. *Proc. ACM Program. Lang.* 10, POPL, Article 34 (Jan. 2026), 29 pages. doi:10.1145/3776676

[22] Pietro Di Gianantonio and Marino Miculan. 2003. A Unifying Approach to Recursive and Co-recursive Definitions. In *Types for Proofs and Programs*, Herman Geuvers and Freek Wiedijk (Eds.). Springer Berlin Heidelberg, Berlin, Heidelberg, 148–161.

[23] Derek Dreyer, Amal Ahmed, and Lars Birkedal. 2011. Logical Step-Indexed Logical Relations. *Log. Methods Comput. Sci.* 7, 2 (2011). doi:10.2168/LMCS-7(2:16)2011

[24] Dan Frumin, Amin Timany, and Lars Birkedal. 2024. Modular Denotational Semantics for Effects with Guarded Interaction Trees. *Proc. ACM Program. Lang.* 8, POPL (2024), 332–361. doi:10.1145/3632854

[25] Vladimir Gladshtein, George Pîrlea, Qiyuan Zhao, Vitaly Kurin, and Ilya Sergey. 2026. Foundational Multi-Modal Program Verifiers. *Proc. ACM Program. Lang.* 10, POPL, Article 77 (Jan. 2026), 32 pages. doi:10.1145/3776719

[26] Simon Oddershede Gregersen, Alejandro Aguirre, Philipp G. Haselwarter, Joseph Tassarotti, and Lars Birkedal. 2024. Asynchronous Probabilistic Couplings in Higher-Order Separation Logic. *Proc. ACM Program. Lang.* 8, POPL (2024), 753–784. doi:10.1145/3632868

[27] Chris Hawblitzel, Jon Howell, Manos Kapritsos, Jacob R. Lorch, Bryan Parno, Michael Lowell Roberts, Srinath T. V. Setty, and Brian Zill. 2015. IronFleet: Proving Practical Distributed Systems Correct. In *Proceedings of the 25th Symposium on Operating Systems Principles, SOSP 2015, Monterey, CA, USA, October 4-7, 2015*. 1–17. doi:10.1145/2815400.2815428

[28] Maurice P. Herlihy and Jeannette M. Wing. 1990. Linearizability: A Correctness Condition for Concurrent Objects. *ACM Trans. Program. Lang. Syst.* 12, 3 (July 1990), 463–492. doi:10.1145/78969.78972

[29] Daniel Hillerström and Sam Lindley. 2016. Liberating Effects with Rows and Handlers. In *Proceedings of the 1st International Workshop on Type-Driven Development* (Nara, Japan) *(TyDe 2016)*. Association for Computing Machinery, New York, NY, USA, 15–27. doi:10.1145/2976022.2976033

[30] Joseph Izraelevitz, Hammurabi Mendes, and Michael L. Scott. 2016. Linearizability of Persistent Memory Objects Under a Full-System-Crash Failure Model. In *Distributed Computing - 30th International Symposium, DISC 2016, Paris, France, September 27-29, 2016. Proceedings (Lecture Notes in Computer Science, Vol. 9888)*, Cyril Gavoille and David Ilcinkas (Eds.). Springer, 313–327. doi:10.1007/978-3-662-53426-7_23

[31] Ralf Jung, Robbert Krebbers, Jacques-Henri Jourdan, Ales Bizjak, Lars Birkedal, and Derek Dreyer. 2018. Iris from the Ground Up: A Modular Foundation for Higher-Order Concurrent Separation Logic. *J. Funct. Program.* 28 (2018), e20. doi:10.1017/S0956796818000151

[32] Ralf Jung, Rodolphe Lepigre, Gaurav Parthasarathy, Marianna Rapoport, Amin Timany, Derek Dreyer, and Bart Jacobs. 2020. The Future Is Ours: Prophecy Variables in Separation Logic. *Proc. ACM Program. Lang.* 4, POPL (2020), 45:1–45:32. doi:10.1145/3371113

[33] Jan-Oliver Kaiser, Hoang-Hai Dang, Derek Dreyer, Ori Lahav, and Viktor Vafeiadis. 2017. Strong Logic for Weak Memory: Reasoning About Release-Acquire Consistency in Iris. In *31st European Conference on Object-Oriented Programming, ECOOP 2017, June 19-23, 2017, Barcelona, Spain*. 17:1–17:29. doi:10.4230/LIPICS.ECOOP.2017.17

[34] Morten Krogh-Jespersen, Amin Timany, Marit Edna Ohlenbusch, Simon Oddershede Gregersen, and Lars Birkedal. 2020. Aneris: A Mechanised Logic for Modular Reasoning about Distributed Systems. In *Programming Languages and Systems - 29th European Symposium on Programming, ESOP 2020, Held as Part of the European Joint Conferences on Theory and Practice of Software, ETAPS 2020, Dublin, Ireland, April 25-30, 2020, Proceedings*. 336–365. doi:10.1007/978-3-030-44914-8_13

[35] Ori Lahav and Viktor Vafeiadis. 2015. Owicki-Gries Reasoning for Weak Memory Models. In *Automata, Languages, and Programming - 42nd International Colloquium, ICALP 2015, Kyoto, Japan, July 6-10, 2015, Proceedings, Part II*. 311–323. doi:10.1007/978-3-662-47666-6_25

[36] Richard J. Lipton. 1975. Reduction: A New Method of Proving Properties of Systems of Processes. In *Conference Record of the Second ACM Symposium on Principles of Programming Languages, Palo Alto, California, USA, January 1975*, Robert M. Graham, Michael A. Harrison, and John C. Reynolds (Eds.). ACM Press, 78–86. doi:10.1145/512976.512985

[37] Martin Hugo Löb. 1955. Solution of a Problem of Leon Henkin. *Journal of Symbolic Logic* 20, 2 (1955), 115–118. doi:10.2307/2266895

[38] Kenji Maillard, Danel Ahman, Robert Atkey, Guido Martínez, Catalin Hritcu, Exequiel Rivas, and Éric Tanter. 2019. Dijkstra Monads for All. *Proc. ACM Program. Lang.* 3, ICFP (2019), 104:1–104:29. doi:10.1145/3341708

[39] Yusuke Matsushita and Takeshi Tsukada. 2025. Nola: Later-Free Ghost State for Verifying Termination in Iris. *Proc. ACM Program. Lang.* 9, PLDI (2025), 98–124. doi:10.1145/3729250

[40] Glen Mével and Jacques-Henri Jourdan. 2021. Formal Verification of a Concurrent Bounded Queue in a Weak Memory Model. *Proc. ACM Program. Lang.* 5, ICFP, Article 66 (Aug. 2021), 29 pages. doi:10.1145/3473571

[41] Glen Mével, Jacques-Henri Jourdan, and François Pottier. 2019. Time Credits and Time Receipts in Iris. In *Programming Languages and Systems*, Luís Caires (Ed.). Springer International Publishing, Cham, 3–29.

[42] Hiroshi Nakano. 2000. A Modality for Recursion. In *15th Annual IEEE Symposium on Logic in Computer Science, Santa Barbara, California, USA, June 26-29, 2000*. 255–266. doi:10.1109/LICS.2000.855774

[43] Gian Ntzik, Pedro da Rocha Pinto, and Philippa Gardner. 2015. Fault-Tolerant Resource Reasoning. In *Programming Languages and Systems - 13th Asian Symposium, APLAS 2015, Pohang, South Korea, November 30 - December 2, 2015, Proceedings*. 169–188. doi:10.1007/978-3-319-26529-2_10

[44] Peter W. O'Hearn. 2007. Resources, Concurrency, and Local Reasoning. *Theor. Comput. Sci.* 375, 1-3 (2007), 271–307. doi:10.1016/j.tcs.2006.12.035

[45] Susan S. Owicki and David Gries. 1976. An Axiomatic Proof Technique for Parallel Programs I. *Acta Informatica* 6 (1976), 319–340. doi:10.1007/BF00268134

[46] Gordon D. Plotkin and John Power. 2001. Adequacy for Algebraic Effects. In *Foundations of Software Science and Computation Structures, 4th International Conference, FOSSACS 2001 Held as Part of the Joint European Conferences on Theory and Practice of Software, ETAPS 2001 Genova, Italy, April 2-6, 2001, Proceedings*. 1–24. doi:10.1007/3-540-45315-6_1

[47] Gordon D. Plotkin and Matija Pretnar. 2009. Handlers of Algebraic Effects. In *Programming Languages and Systems, 18th European Symposium on Programming, ESOP 2009, Held as Part of the Joint European Conferences on Theory and Practice of Software, ETAPS 2009, York, UK, March 22-29, 2009. Proceedings*. 80–94. doi:10.1007/978-3-642-00590-9_7

[48] Azalea Raad, Ori Lahav, and Viktor Vafeiadis. 2020. Persistent Owicki-Gries Reasoning: A Program Logic for Reasoning about Persistent Programs on Intel-x86. *Proc. ACM Program. Lang.* 4, OOPSLA (2020), 151:1–151:28. doi:10.1145/3428219

[49] John C. Reynolds. 2002. Separation Logic: A Logic for Shared Mutable Data Structures. In *17th IEEE Symposium on Logic in Computer Science (LICS 2002), 22-25 July 2002, Copenhagen, Denmark, Proceedings*. 55–74. doi:10.1109/LICS.2002.1029817

[50] Upamanyu Sharma, Ralf Jung, Joseph Tassarotti, M. Frans Kaashoek, and Nickolai Zeldovich. 2023. Grove: A Separation-Logic Library for Verifying Distributed Systems. In *Proceedings of the 29th Symposium on Operating Systems Principles, SOSP 2023, Koblenz, Germany, October 23-26, 2023*. 113–129. doi:10.1145/3600006.3613172

[51] Simon Spies, Lennard Gäher, Joseph Tassarotti, Ralf Jung, Robbert Krebbers, Lars Birkedal, and Derek Dreyer. 2022. Later Credits: Resourceful Reasoning for the Later Modality. *Proc. ACM Program. Lang.* 6, ICFP, Article 100 (Aug. 2022), 29 pages. doi:10.1145/3547631

[52] Nikhil Swamy, Joel Weinberger, Cole Schlesinger, Juan Chen, and Benjamin Livshits. 2013. Verifying Higher-Order Programs with the Dijkstra Monad. In *ACM SIGPLAN Conference on Programming Language Design and Implementation, PLDI '13, Seattle, WA, USA, June 16-19, 2013*. 387–398. doi:10.1145/2491956.2491978

[53] Wenhao Tang, Leo White, Stephen Dolan, Daniel Hillerström, Sam Lindley, and Anton Lorenzen. 2025. Modal Effect Types. *Proc. ACM Program. Lang.* 9, OOPSLA1, Article 120 (April 2025), 28 pages. doi:10.1145/3720476

[54] Joseph Tassarotti, Tej Chajed, and Perennial contributors. 2021. Crash Modality in Perennial's Mechanization. https://github.com/mit-pdos/perennial/blob/c4c806b6ede0580dc8cb72ca873d25e3fb7e564d/src/goose_lang/crash_modality.v

[55] Joseph Tassarotti and Robert Harper. 2019. A Separation Logic for Concurrent Randomized Programs. *Proc. ACM Program. Lang.* 3, POPL (2019), 64:1–64:30. doi:10.1145/3290377

[56] Joseph Tassarotti and Perennial contributors. 2022. Crash Borrow in Perennial's Mechanization. https://github.com/mit-pdos/perennial/blob/5189a4eccc8583a7042ccccbbb3bb13cfb57e4c5/src/goose_lang/crash_borrow.v

[57] Amin Timany, Robbert Krebbers, Derek Dreyer, and Lars Birkedal. 2024. A Logical Approach to Type Soundness. *J. ACM* 71, 6 (2024), 40:1–40:75. doi:10.1145/3676954

[58] Aaron Turon, Derek Dreyer, and Lars Birkedal. 2013. Unifying Refinement and Hoare-Style Reasoning in a Logic for Higher-Order Concurrency. In *ACM SIGPLAN International Conference on Functional Programming, ICFP'13, Boston, MA, USA - September 25 - 27, 2013*. 377–390. doi:10.1145/2500365.2500600

[59] Aaron Turon, Viktor Vafeiadis, and Derek Dreyer. 2014. GPS: Navigating Weak Memory with Ghosts, Protocols, and Separation. In *Proceedings of the 2014 ACM International Conference on Object Oriented Programming Systems Languages & Applications, OOPSLA 2014, part of SPLASH 2014, Portland, OR, USA, October 20-24, 2014*. 691–707. doi:10.1145/2660193.2660243

[60] Viktor Vafeiadis and Chinmay Narayan. 2013. Relaxed Separation Logic: A Program Logic for C11 Concurrency. In *Proceedings of the 2013 ACM SIGPLAN International Conference on Object Oriented Programming Systems Languages & Applications, OOPSLA 2013, part of SPLASH 2013, Indianapolis, IN, USA, October 26-31, 2013*. 867–884. doi:10.1145/2509136.2509532

[61] Orpheas van Rooij and Robbert Krebbers. 2025. Affect: An Affine Type and Effect System. *Proc. ACM Program. Lang.* 9, POPL, Article 5 (Jan. 2025), 29 pages. doi:10.1145/3704841

[62] Simon Friis Vindum, Aïna Linn Georges, and Lars Birkedal. 2025. The Nextgen Modality: A Modality for Non-Frame-Preserving Updates in Separation Logic. In *Proceedings of the 14th ACM SIGPLAN International Conference on Certified Programs and Proofs* (Denver, CO, USA) *(CPP '25)*. Association for Computing Machinery, New York, NY, USA, 83–97. doi:10.1145/3703595.3705876

[63] Max Vistrup, Klaus Kraßnitzer, Michael Sammler, and Ralf Jung. 2026. Mosaic: From Language Fragments to Program Logics. https://maxv.dk/mosaic.pdf

[64] Max Vistrup, Michael Sammler, and Ralf Jung. 2025. Program Logics à la Carte. *Proc. ACM Program. Lang.* 9, POPL (2025), 300–331. doi:10.1145/3704847

[65] James R. Wilcox, Ilya Sergey, and Zachary Tatlock. 2017. Programming Language Abstractions for Modularly Verified Distributed Systems. In *2nd Summit on Advances in Programming Languages, SNAPL 2017, May 7-10, 2017, Asilomar, CA, USA*. 19:1–19:12. doi:10.4230/LIPICS.SNAPL.2017.19

[66] James R. Wilcox, Doug Woos, Pavel Panchekha, Zachary Tatlock, Xi Wang, Michael D. Ernst, and Thomas E. Anderson. 2015. Verdi: A Framework for Implementing and Formally Verifying Distributed Systems. In *Proceedings of the 36th ACM SIGPLAN Conference on Programming Language Design and Implementation, Portland, OR, USA, June 15-17, 2015*. 357–368. doi:10.1145/2737924.2737958

[67] Li-yao Xia, Yannick Zakowski, Paul He, Chung-Kil Hur, Gregory Malecha, Benjamin C. Pierce, and Steve Zdancewic. 2020. Interaction Trees: Representing Recursive and Impure Programs in Coq. *Proc. ACM Program. Lang.* 4, POPL (2020), 51:1–51:32. doi:10.1145/3371119

[68] Zichen Zhang, Simon Oddershede Gregersen, and Joseph Tassarotti. 2026. *Building Extensible Program Logics through Effect Handlers (Artifact)*. doi:10.5281/zenodo.23234723

A Ficus

A.1 Semantics

Head Reduction Rules. $e \xrightarrow{\vec{\kappa}}_{\mathsf{h}} e'$

$$\textsc{Hd-Beta}\quad (\mathbf{rec}\ f\ x.\ e)\ v \xrightarrow{\epsilon}_{\mathsf{h}} e[(\mathbf{rec}\ f\ x.\ e), v/f, x] \qquad \textsc{Hd-Cont}\quad (\mathbf{cont}\ N)\ v \xrightarrow{\epsilon}_{\mathsf{h}} N[v]$$

$$\textsc{Hd-EffApR}\quad e_1\ \S(N)[v_2] \xrightarrow{\epsilon}_{\mathsf{h}} \S(e_1\ N)[v_2] \qquad \textsc{Hd-Do}\quad \mathbf{do}\ v \xrightarrow{\epsilon}_{\mathsf{h}} \S(\bullet)[v]$$

$$\textsc{Hd-EffApL}\quad \S(N)[v_1]\ v_2 \xrightarrow{\epsilon}_{\mathsf{h}} \S(N\ v_2)[v_1] \qquad \textsc{Hd-Pick}\quad \mathbf{pick} \xrightarrow{\epsilon}_{\mathsf{h}} b$$

$$\textsc{Hd-EffDo}\quad \mathbf{do}\ \S(N)[v] \xrightarrow{\epsilon}_{\mathsf{h}} \S(\mathbf{do}\ N)[v] \qquad \textsc{Hd-Obs}\quad \mathbf{observe}\ v \xrightarrow{[v]}_{\mathsf{h}} ()$$

$$\textsc{Hd-TryEff}\quad \mathbf{try}\ \S(N)[v_0]\ \mathbf{with}\ v_1\ k \Rightarrow e_1 \mid \mathbf{ret}\ v_2 \Rightarrow e_2 \xrightarrow{\epsilon}_{\mathsf{h}} e_1[v_0, \mathbf{cont}\ N/v_1, k]$$

$$\textsc{Hd-TryVal}\quad \mathbf{try}\ v_0\ \mathbf{with}\ v_1\ k \Rightarrow e_1 \mid \mathbf{ret}\ v_2 \Rightarrow e_2 \xrightarrow{\epsilon}_{\mathsf{h}} e_2[v_0/v_2]$$

Pure Reduction and Its Reflexive Transitive Closure. $e \xrightarrow{\vec{\kappa}} e'$ and $e \xrightarrow{\vec{\kappa}}{}^{*} e'$

$$e \xrightarrow{\vec{\kappa}} e' \triangleq \exists K, \tilde{e}, \tilde{e}'.\ e = K[\tilde{e}] \wedge e' = K[\tilde{e}'] \wedge \tilde{e} \xrightarrow{\vec{\kappa}}_{\mathsf{h}} \tilde{e}'$$

$$e \xrightarrow{\vec{\kappa}}{}^{*} e' \triangleq (e = e' \wedge \vec{\kappa} = \epsilon) \vee (\exists e'', \vec{\kappa}_1, \vec{\kappa}_2.\ \vec{\kappa} = \vec{\kappa}_1 +\!\!+ \vec{\kappa}_2 \wedge e \xrightarrow{\vec{\kappa}_1} e'' \wedge e'' \xrightarrow{\vec{\kappa}_2}{}^{*} e')$$

A.2 Reasoning Rules

$$\textsc{Ewp-Value}\ \frac{{}_{W_1}\!\Rrightarrow_{W_2} \Phi(v)}{\mathsf{ewp}_{W_1,W_2}\ v\ \langle\Psi\rangle\ \{\Phi\}}* \qquad \textsc{Ewp-Do}\ \frac{\Psi(v, \Phi)}{\mathsf{ewp}_{W}\ \mathbf{do}\ v\ \langle\Psi\rangle\ \{\Phi\}}* \qquad \textsc{Ewp-DoWupd}\ \frac{{}_{W_1}\!\Rrightarrow_{\bot} \Psi(v, (\lambda r.\ {}_{\bot}\!\Rrightarrow_{W_2} \Phi(r)))}{\mathsf{ewp}_{W_1,W_2}\ \mathbf{do}\ v\ \langle\Psi\rangle\ \{\Phi\}}*$$

$$\textsc{Ewp-Mono}\ \frac{\mathsf{ewp}_{W_1,W_2}\ e\ \langle\Psi\rangle\ \{\Phi\} \qquad \Psi \sqsubseteq \Psi' \qquad \forall v.\ \Phi(v) \mathrel{-\!\!*} {}_{W_2}\!\Rrightarrow \Phi'(v)}{\mathsf{ewp}_{W_1,W_2}\ e\ \langle\Psi'\rangle\ \{\Phi'\}}* \qquad \textsc{Ewp-WorldMono}\ \frac{\mathsf{ewp}_{W}\ e\ \langle\Psi\rangle\ \{\Phi\} \qquad W \sqsubseteq W'}{\mathsf{ewp}_{W'}\ e\ \langle\Psi\rangle\ \{\Phi\}}*$$

$$\textsc{Ewp-Frame}\ \frac{R \qquad \mathsf{ewp}_{W_1,W_2}\ e\ \langle\Psi\rangle\ \{\Phi\}}{\mathsf{ewp}_{W_1,W_2}\ e\ \langle\Psi\rangle\ \{v.\ R * \Phi(v)\}}* \qquad \textsc{Ewp-WupdPre}\ \frac{{}_{W_1}\!\Rrightarrow_{W_2} \mathsf{ewp}_{W_2,W_3}\ e\ \langle\Psi\rangle\ \{\Phi\}}{\mathsf{ewp}_{W_1,W_3}\ e\ \langle\Psi\rangle\ \{\Phi\}}*$$

$$\textsc{Ewp-WupdPost}\ \frac{\mathsf{ewp}_{W_1,W_2}\ e\ \langle\Psi\rangle\ \{v.\ {}_{W_2}\!\Rrightarrow_{W_3} \Phi(v)\}}{\mathsf{ewp}_{W_1,W_3}\ e\ \langle\Psi\rangle\ \{\Phi\}}* \qquad \textsc{Ewp-Step}\ \frac{\forall e'.\ e \rightarrow e' \mathrel{-\!\!*} \mathsf{ewp}\ e'\ \langle\Psi\rangle\ \{\Phi\} \qquad \exists e'.\ e \rightarrow e'}{\mathsf{ewp}\ e\ \langle\Psi\rangle\ \{\Phi\}}*$$

$$\textsc{Ewp-Bind}\ \frac{\mathsf{ewp}_{W_1,W_2}\ e\ \langle\Psi\rangle\ \{v.\ \mathsf{ewp}_{W_2,W_3}\ N[v]\ \langle\Psi\rangle\ \{\Phi\}\}}{\mathsf{ewp}_{W_1,W_3}\ N[e]\ \langle\Psi\rangle\ \{\Phi\}}* \qquad \textsc{Ewp-PureBind}\ \frac{\mathsf{ewp}_{W_1,W_2}\ e\ \langle\bot\rangle\ \{v.\ \mathsf{ewp}_{W_2,W_3}\ K[v]\ \langle\Psi\rangle\ \{\Phi\}\}}{\mathsf{ewp}_{W_1,W_3}\ K[e]\ \langle\Psi\rangle\ \{\Phi\}}*$$

$$\textsc{Ewp-WorldFrame}\ \frac{\mathsf{ewp}_{W_1,W_2}\ e\ \langle\Psi\rangle\ \{\Phi\}}{\mathsf{ewp}_{W_1\oplus W,W_2\oplus W}\ e\ \langle\Psi\rangle\ \{\Phi\}}* \qquad \textsc{Ewp-Obs}\ \frac{\mathsf{primProph}(\vec{v})}{\mathsf{ewp}_{W}\ \mathbf{observe}\ w\ \langle\Psi\rangle\ \{_.\ \exists \vec{v}'.\ \vec{v} = w :: \vec{v}' * \mathsf{primProph}(\vec{v}')\}}*$$

A.3 Model

$$\mathsf{ewp}_{W_1,W_2}\ v\ \langle\Psi\rangle\ \{\Phi\} \triangleq {}_{W_1}\!\Rrightarrow_{W_2} \Phi(v)$$

$$\mathsf{ewp}_{W_1,W_2}\ \S(\bullet)[v]\ \langle\Psi\rangle\ \{\Phi\} \triangleq {}_{W_1}\!\Rrightarrow_{\bot} \Psi(v, \lambda w.\ {}_{\bot}\!\Rrightarrow_{W_2} \Phi(w))$$

$$\mathsf{ewp}_{\mathrm{W}_1,\mathrm{W}_2}\ \S(N)[v]\ \langle\Psi\rangle\ \{\Phi\} \triangleq {}_{\mathrm{W}_1}\Rrightarrow_{\bot}\Psi(v, \lambda w.\ \Rrightarrow_{\bot} \triangleright(\mathsf{ewp}_{\bot,\mathrm{W}_2}\ N[w]\ \langle\Psi\rangle\ \{\Phi\}))$$

$$\mathsf{ewp}_{\mathrm{W}_1,\mathrm{W}_2}\ e\ \langle\Psi\rangle\ \{\Phi\} \triangleq \forall\vec{\kappa}_1,\vec{\kappa}_2.\ \mathsf{primProph}_\bullet(\vec{\kappa}_1 +\!\!+ \vec{\kappa}_2) \mathrel{-\!\!*} {}_{\mathrm{W}_1}\Rrightarrow_{\bot} \mathsf{red}(e) *$$
$$\forall e'.\ e \xrightarrow{\vec{\kappa}_1} e' \mathrel{-\!\!*} \pounds(1) \mathrel{-\!\!*} \Rrightarrow_{\bot} \triangleright \Rrightarrow_{\bot} \mathsf{primProph}_\bullet(\vec{\kappa}_2) * \mathsf{ewp}_{\bot,\mathrm{W}_2}\ e'\ \langle\Psi\rangle\ \{\Phi\}$$

$$\mathsf{primProph}_\bullet(\vec{v}) \triangleq \boxed{\bullet\mathsf{ex}(\vec{v})}^{\gamma_p} \qquad \mathsf{primProph}(\vec{v}) \triangleq \boxed{\circ\mathsf{ex}(\vec{v})}^{\gamma_p}$$

B Protocols for the Prophetic Heap

$$\mathsf{PROPH_ALLOC}(v,\Phi) \triangleq \exists x.\ v = (\mathtt{alloc}, x) * (\forall \ell, \vec{v}.\ \ell \mapsto x * \mathsf{proph}(\ell,\vec{v}) * \mathsf{prophE}(\ell) \mathrel{-\!\!*} \Phi(\ell))$$

$$\mathsf{PROPH_LOAD}(v,\Phi) \triangleq \exists \ell, x, \vec{v}.\ v = (\mathtt{load}, \ell) * \ell \mapsto x * \mathsf{proph}(\ell,\vec{v}) * \mathsf{prophE}(\ell) *$$
$$(\forall \vec{v}'.\ \ell \mapsto x * \vec{v} = (x, \mathsf{load}) :: \vec{v}' * \mathsf{proph}(\ell,\vec{v}') * \mathsf{prophE}(\ell) \mathrel{-\!\!*} \Phi(x))$$

$$\mathsf{PROPH_LOAD'}(v,\Phi) \triangleq \exists \ell, q, x.\ v = (\mathtt{load}, \ell) * \ell \overset{q}{\mapsto} x * \mathsf{prophD}(\ell) * (\ell \overset{q}{\mapsto} x \mathrel{-\!\!*} \Phi(x))$$

$$\mathsf{PROPH_STORE}(v,\Phi) \triangleq \exists \ell, x, y, \vec{v}.\ v = (\mathtt{store}, (\ell, y)) * \ell \mapsto x * \mathsf{proph}(\ell,\vec{v}) * \mathsf{prophE}(\ell) *$$
$$(\forall \vec{v}'.\ \ell \mapsto y * \vec{v} = ((), \mathsf{store}(y)) :: \vec{v}' * \mathsf{proph}(\ell,\vec{v}') * \mathsf{prophE}(\ell) \mathrel{-\!\!*} \Phi(()))$$

$$\mathsf{PROPH_CAS}(v,\Phi) \triangleq \exists \ell, w, x, y, \vec{v}.\ v = (\mathtt{cas}, (\ell, x, y)) * \ell \mapsto w * \mathsf{proph}(\ell,\vec{v}) * \mathsf{prophE}(\ell) *$$
$$(\forall \vec{v}'.\ \ell \mapsto (\text{if } w = x \text{ then } y \text{ else } w) * \vec{v} = ((w, w = x), \mathsf{cas}(x, y)) :: \vec{v}' *$$
$$\mathsf{proph}(\ell,\vec{v}') * \mathsf{prophE}(\ell) \mathrel{-\!\!*} \Phi((w, w = x)))$$

$$\mathsf{PROPH_CAS'}(v,\Phi) \triangleq \exists \ell, q, w, x, y.\ v = (\mathtt{cas}, (\ell, x, y)) * \ell \overset{q}{\mapsto} w * w \neq x * \mathsf{prophD}(\ell) *$$
$$(\ell \overset{q}{\mapsto} w \mathrel{-\!\!*} \Phi((w, \mathtt{false})))$$

Each location is associated with a prophecy variable $\mathsf{proph}(\ell,\vec{v})$ and a pair of tokens $\mathsf{prophE}(\ell)$/$\mathsf{prophD}(\ell)$ controlling whether the prophecy variable is enabled or disabled. The prophecy variable is enabled when the location is allocated, and one can disable it using rule $\mathsf{prophE}(\ell) \vdash \Rrightarrow \mathsf{prophD}(\ell)$. Once disabled, a location can no longer be enabled. In fact, token $\mathsf{prophD}(\ell)$ merely carries the knowledge that one agrees to never use the prophecy variable associated with the location and it is therefore persistent. To perform an operation on a location, one must either provide the ownership of the associated prophecy variable and the enable token, or provide the disable token.

C Details about the Crash Recovery System

This section uses the complete $\mathrm{CC}(\mathcal{E}, R_c)$ token. Compared to $\mathrm{CC}(R_c)$ used in §6, it has one extra parameter for the enabled crash borrows. The connection between the two tokens is $\mathrm{CC}(R_c) = \mathrm{CC}(\top, R_c)$.

C.1 Post-crash Modality

$$\mathsf{crashed} \triangleq \exists M : \mathit{PID} \to \mathit{List}(\mathit{Val} \times \mathit{Val}).\ \boxed{\bullet M}^{\gamma_{\mathsf{proph}}} * \forall p \mapsto \vec{v} \in M.\ \vec{v} = \epsilon$$
$$\blacklozenge P \triangleq \mathsf{crashed} \mathrel{-\!\!*} \mathsf{crashed} * P$$

C.2 Protocol

$$\mathsf{CRASH}(v,\Phi) \triangleq v = (\mathtt{crash}, ()) * (\Rrightarrow \mathcal{R})$$
$$\mathsf{DURA}_{\mathrm{W}}(\Psi)(v,\Phi) \triangleq \exists R.\ \Psi(v, (\lambda r.\ {}_{\bot}\Rrightarrow_{\mathrm{W}\oplus\mathrm{CC}(\top,R)} (R \wedge {}_{\mathrm{W}\oplus\mathrm{CC}(\top,R)}\Rrightarrow_{\bot} \Phi(r))))$$
$$\mathsf{CRASHCONC}_{\mathrm{W}}(\Psi) \triangleq \mathsf{DURA}_{\mathrm{W}}(\Psi \oplus \mathsf{FORK}'_{\mathrm{W}}(\Psi))$$

$$\mathsf{FORK}'_{\mathrm{W}}(\Psi)(v,\Phi) \triangleq \exists e.\, v = (\mathtt{fork}, \lambda_.\, e) *$$
$$\triangleright\, \mathsf{ewp}_{\mathrm{W}\oplus\mathsf{CC}(\top,\mathsf{True}),\bot}\ e\ \langle \mathsf{CRASHCONC}_{\mathrm{W}}(\Psi)\rangle\ \left\{_.\ \exists R_e.\ \bot \Rrightarrow_{\mathrm{W}\oplus\mathsf{CC}(\top,R_e)} R_e\right\} * \Phi(())$$

In the CRASH protocol, $\mathcal{R}$ is the global crash invariant, and there is no requirement on Φ because crash will never return. CRASHCONC intentionally uses the same tag as the regular CONC protocol to prevent having two schedulers. The FORK′ protocol permits a child thread to change the crash condition during execution, as long as it is consistent with the $\mathsf{CC}(\mathcal{E}, R)$ token. The crash condition cannot be violated even if a thread terminates.

The relations between these protocols are

$$\mathsf{DURA}_{\mathrm{W}}(\Psi) \sqsubseteq \mathsf{ATOM}_{\mathrm{W}}(\Psi) \sqsubsetneq \Psi \qquad\qquad \mathsf{ATOM}_{\mathrm{W}}(\Psi) \sqsubseteq \mathsf{CONC}_{\mathrm{W}}(\Psi)$$

$$\mathsf{DURA}_{\mathrm{W}}(\Psi) \sqsubseteq \mathsf{CRASHCONC}_{\mathrm{W}}(\Psi)$$

Ewp-Seq-Atom
$$\frac{\mathsf{ewp}_{\mathrm{W}_I}\ e\ \langle\Psi\rangle\ \{\Phi\}}{\mathsf{ewp}_{\mathrm{W}_I\oplus\mathrm{W}}\ e\ \langle \mathsf{ATOM}_{\mathrm{W}}(\Psi)\rangle\ \{\Phi\}}*$$

Ewp-Atom-Dura
$$\frac{\Box(R' \mathrel{-\!\!*} R) \qquad R' \qquad \mathsf{ewp}_{\mathrm{W}}\ e\ \langle \mathsf{ATOM}_{\mathrm{W}}(\Psi)\rangle\ \{v.\, R' \mathrel{-\!\!*} \Phi(v)\}}{\mathsf{ewp}_{\mathrm{W}\oplus\mathsf{CC}(\top,R)}\ e\ \langle \mathsf{DURA}_{\mathrm{W}}(\Psi)\rangle\ \{\Phi\}}*$$

Ewp-Conc-Crashconc
$$\frac{\mathsf{ewp}_{\mathrm{W}}\ e\ \langle \mathsf{CONC}_{\mathrm{W}}(\Psi)\rangle\ \{\Phi\}}{\mathsf{ewp}_{\mathrm{W}\oplus\mathsf{CC}(\top,\mathsf{True})}\ e\ \langle \mathsf{CRASHCONC}_{\mathrm{W}}(\Psi)\rangle\ \{\Phi\}}*$$

C.3 Crash Hoare Logic

$$\mathsf{ewpc}^0_{(\mathcal{E}_1,R_1),(\mathcal{E}_2,R_2)}\ e\ \langle\Psi\rangle\ \{\Phi\} \triangleq \mathsf{ewp}_{\mathrm{W}_1,\mathrm{W}_2}\ e\ \left\langle \mathsf{CRASHCONC}_{\mathsf{TokI}(\top)}(\Psi)\right\rangle\ \{\Phi\}$$
$$\text{where} \quad \mathrm{W}_i \triangleq \mathsf{TokI}(\mathcal{E}_i) \oplus \mathsf{CC}(\mathcal{E}_i, R_i)$$

The ewpc^0 assertion does not support the monotonicity rule, but this will not become a restriction in practice because one can always use the upward closure of a non-mask-changing ewpc^0.

$$\mathsf{ewpc}_{(\mathcal{E},R)}\ e\ \langle\Psi\rangle\ \{\Phi\} \triangleq \forall R', \Phi'.\ (\forall v.\, \Phi(v) \mathrel{-\!\!*} \Phi'(v)) \wedge (R \mathrel{-\!\!*} R') \mathrel{-\!\!*} \mathsf{ewpc}^0_{(\mathcal{E},R')}\ e\ \langle\Psi\rangle\ \{\Phi'\}$$

The connection between Φ and R is a logical conjunction $\wedge$ because only one part of this conjunction is needed at a time. The Φ part is used during normal execution and the R part is used when the system crashes.

C.4 Crash Borrow

Client Rules.

Wupd-CBrwAlloc
$$\frac{\triangleright P \qquad \Box(\triangleright P \mathrel{-\!\!*} \triangleright R)}{\mathsf{CC}(\mathcal{E},R_c * R) \Rrightarrow_{\mathsf{CC}(\mathcal{E},R_c)} \boxed{P \mid R}^{\mathcal{N}}}*$$

Wupd-CBrwReturn
$$\frac{\boxed{P \mid R}^{\mathcal{N}} \qquad \mathcal{N} \subseteq \mathcal{E}}{\mathsf{CC}(\mathcal{E},R_c) \Rrightarrow_{\mathsf{CC}(\mathcal{E},R_c * R)} \triangleright P}*$$

Wupd-CBrwRename
$$\frac{\boxed{P \mid R}^{\mathcal{N}} \qquad \mathcal{N} \subseteq \mathcal{E}}{\Rrightarrow_{\mathsf{CC}(\mathcal{E},R_c)} \boxed{P \mid R}^{\mathcal{N}}}*$$

Wupd-CBrwAccUpdate
$$\frac{\boxed{P \mid R}^{\mathcal{N}} \qquad \mathcal{N} \subseteq \mathcal{E}}{\mathsf{CC}(\mathcal{E},R_c) \Rrightarrow_{\mathsf{CC}(\mathcal{E}\setminus\mathcal{N},R_c)} \triangleright P * \left(\forall Q.\ \triangleright Q * \Box(\triangleright Q \mathrel{-\!\!*} \triangleright R) \mathrel{-\!\!*} \mathsf{CC}(\mathcal{E}\setminus\mathcal{N},R_c) \Rrightarrow_{\mathsf{CC}(\mathcal{E},R_c)} \boxed{Q \mid R}^{\mathcal{N}}\right)}*$$

Wupd-CBrwMono
$$\frac{\boxed{P \mid R}^{\mathcal{N}} \qquad \triangleright \Box(P' \mathrel{-\!\!*} R') \qquad \triangleright (P \mathrel{-\!\!*} P') \qquad \triangleright \Box(R' \mathrel{-\!\!*} R) \qquad \mathcal{N} \subseteq \mathcal{E}}{\Rrightarrow_{\mathsf{CC}(\mathcal{E},R_c)} \boxed{P' \mid R'}^{\mathcal{N}}}*$$

$$\textsc{Wupd-CBrwSplit}\quad \frac{\boxed{P_1 * P_2 \mid R_1 * R_2}^{\mathcal{N}} \qquad \square(\triangleright P_1 \mathrel{-\!\!*} \triangleright R_1) \qquad \square(\triangleright P_2 \mathrel{-\!\!*} \triangleright R_2) \qquad \mathcal{N} \subseteq \mathcal{E}}{\Rightarrow_{\mathrm{CC}(\mathcal{E},R_c)} \boxed{P_1 \mid R_1}^{\mathcal{N}} * \boxed{P_2 \mid R_2}^{\mathcal{N}}}*$$

$$\textsc{Wupd-CBrwCombine}\quad \frac{\boxed{P_1 \mid R_1}^{\mathcal{N}} \qquad \boxed{P_2 \mid R_2}^{\mathcal{N}} \qquad \mathcal{N} \subseteq \mathcal{E}}{\Rightarrow_{\mathrm{CC}(\mathcal{E},R_c)} \boxed{P_1 * P_2 \mid R_1 * R_2}^{\mathcal{N}}}*$$

Handler Rules.

$$\textsc{Wsat-CinvAlloc}\quad \vdash \Rrightarrow \exists \gamma_{brw}, \gamma_{cinv}, \gamma_{cinvset}, \gamma_{cond}, \gamma_{active}.\ \mathrm{CC}(\top, \mathcal{R}) * \boxed{\circ\mathrm{ex}(\{\iota\})}^{\gamma_{cinvset}} * \boxed{\bullet\mathrm{ex}(\iota)}^{\gamma_{active}} * \mathrm{CInv}(\mathcal{R})$$

$$\textsc{Wsat-CinvDestruct}\quad \frac{\boxed{\circ\mathrm{ex}(\mathrm{dom}(I))}^{\gamma_{cinvset}} \qquad \ast_{\iota \mapsto R_c \in I} \boxed{\circ\,[\iota \leftarrow \mathrm{ag}(\mathrm{next}(R_c))]}^{\gamma_{cond}} * \triangleright R_c \qquad \mathrm{CInv}(\mathcal{R}) \qquad \mathrm{WCBrw} \qquad \boxed{\top}^{E}}{\Rightarrow \triangleright \mathcal{R}}*$$

Model.

$$\boxed{P \mid R}^{\mathcal{N}} \triangleq \exists i.\ i \in \mathcal{N} * \boxed{\circ\,[i \leftarrow \mathrm{ag}(\mathrm{next}(P, R))]}^{\gamma_{brw}} \qquad \mathrm{CInv}(\mathcal{R}) \triangleq \boxed{\circ\mathrm{ex}(\mathrm{next}(\mathcal{R}))}^{\gamma_{cinv}}$$

$$\mathrm{CC}(\mathcal{E}, R_c) \triangleq \mathrm{WCBrw} \oplus \boxed{\mathcal{E}}^{E} \oplus \left(\exists \iota.\ \boxed{\circ\mathrm{ex}(\iota)}^{\gamma_{active}} * \boxed{\circ\,[\iota \leftarrow \mathrm{ag}(\mathrm{next}(R_c))]}^{\gamma_{cond}}\right)$$

$$\mathrm{WCBrw} \triangleq \exists B\colon \mathbb{N} \xrightarrow{\mathrm{fin}} iProp \times iProp, C\colon \mathbb{N} \xrightarrow{\mathrm{fin}} iProp, \mathcal{R}\colon iProp.$$

$$\boxed{\bullet\mathrm{ag}\ \langle\$\rangle\ \mathrm{next}\ \langle\$\rangle\ B}^{\gamma_{brw}} * \boxed{\bullet\mathrm{ag}\ \langle\$\rangle\ \mathrm{next}\ \langle\$\rangle\ C}^{\gamma_{cond}} * \boxed{\bullet\mathrm{ex}(\mathrm{dom}(C))}^{\gamma_{cinvset}} * \boxed{\bullet\mathrm{ex}(\mathrm{next}(\mathcal{R}))}^{\gamma_{cinv}} *$$

$$\left(\ast_{i \mapsto (P,R) \in B} \left(\triangleright P * \boxed{\{i\}}^{D} \vee \boxed{\{i\}}^{E}\right) * \square(\triangleright P \mathrel{-\!\!*} \triangleright R)\right) * \left(\left(\ast_{(_,R) \in B} \triangleright R\right) * \left(\ast_{R_c \in C} \triangleright R_c\right) \mathrel{-\!\!*} \triangleright \mathcal{R}\right)$$

C.5 Asynchronous Disk

Our mechanization actually uses two forms of disk points-to assertions: $\ell \mapsto^{\mathrm{d}} [v_c]v$ for the asynchronous disk points-to introduced in §6.2, and $\ell \mapsto^{\mathrm{d}} v$ for a points-to on the physical disk. When a crash occurs, the asynchronous points-to collapses into a disk points-to: $\ell \mapsto^{\mathrm{d}} [v_c]v \vdash \blacklozenge \ell \mapsto^{\mathrm{d}} v_c$. When the system recovers from a crash, the recovery procedure will restore disk points-tos to asynchronous points-tos, which is captured by another modality known as *setup modality*: $\ell \mapsto^{\mathrm{d}} v_c \vdash \diamond \ell \mapsto^{\mathrm{d}} [v_c]v_c$. The main body of the paper omits the setup modality and directly uses asynchronous points-tos to describe post-crash disk values.

Use of Prophecy Variables in the Effect Handler. The handler for asynchronous disk is shown in §G.12. The handler uses a volatile state to store the buffer. For each buffered location, the handler associates a prophecy variable to it, indicating whether this location will be written back before crash. For buffer item $\ell \mapsto (v, p)$, assertion willWB defined below prophesies that the value v will be written back.

$$\mathrm{willWB}(\vec{v}) \triangleq \exists \vec{v}'.\ \vec{v} = ((), \mathtt{true}) :: \vec{v}'$$

For an `adisk_load` effect, the handler returns the cached value if location ℓ is in the buffer; otherwise, it uses the **`disk_load`** operation to load the value directly from the physical disk. For an `adisk_store` effect, the handler always writes the result to the buffer, but if the location is already

in the buffer, the handler will resolve the associated prophecy variable to **false**, meaning that the old value in the buffer will never be written back (as it has been overwritten by the new value). For an adisk_barrier effect, the handler writes back the whole buffer and resolves each prophecy variable to **true**. In addition to the explicit barrier, after each load and store, partial_flush non-deterministically writes back an arbitrary subset of the buffered writes, modeling the disk flushing buffers on its own. In the event of a crash, all prophecy variables will become ϵ and because $\epsilon \neq ((), \mathbf{true}) :: _$, we learn that remaining values in the buffer will never be written back.

Concretely, the asynchronous disk points-to assertion $\ell \mapsto^{\mathrm{d}} [v_c]v$ is defined as a view of the $\mathsf{adp}(B, \ell, v_c, v)$ assertion, where B is the buffer.

$$\mathsf{adp}(B, \ell, v_c, v) \triangleq \ell \notin B * \ell \mapsto^{\mathrm{d}} v * v_c = v$$
$$\vee\, \exists p.\, B(\ell) = (v, p) * \exists \vec{v}.\, \mathsf{proph}(p, \vec{v}) * (\text{if } \mathsf{willWB}(\vec{v}) \text{ then } (\exists x.\, \ell \mapsto^{\mathrm{d}} x) * v_c = v \text{ else } \ell \mapsto^{\mathrm{d}} v_c)$$

The assertion consists of two cases. If ℓ is not in the buffer, then v is in the physical disk and $v_c = v$. If ℓ is in the buffer, then v is the buffered value and the value in the physical disk depends on willWB. If willWB, then the current value in the physical disk is unknown but also unimportant because it will eventually become v; otherwise, the value in the physical disk is v_c. The connection between $\mathsf{adp}(B, \ell, v_c, v)$ and $\ell \mapsto^{\mathrm{d}} [v_c]v$ is enforced by the authoritative resource algebra.

D Details about the Node-Local Specification Resource

The spec resource for the IronFleet-style scheduler is defined as

$$\mathsf{specn}^{\gamma}_{t}(e) \triangleq \exists k, r.\, \boxed{\circ\{(k, r, t)\}}^{\gamma} * \mathsf{loop}(k\ r, e)$$
$$\mathsf{loop}(e_0, e) \triangleq (\forall K.\, \mathsf{spec}(K[e_0]) \twoheadrightarrow \Rrightarrow_{\bot} \mathsf{spec}(K[e]))$$
$$\vee\, (\forall K.\, \mathsf{spec}(K[e_0]) \Rrightarrow\!\!\!\!\ast_{\bot} \exists H, t, M.\, \mathsf{spec}(K[\S(H)[(\mathtt{recv}, t)]]) * t \rightsquigarrow^{\mathrm{lb}} M * \mathsf{loop}(H[\mathbf{inl}\ ()], e))$$
$$\vee\, (\forall K.\, \mathsf{spec}(K[e_0]) \Rrightarrow\!\!\!\!\ast_{\bot} \exists H, s, t, m, M.\, \mathsf{spec}(K[\S(H)[(\mathtt{recv}, t)]]) * t \rightsquigarrow^{\mathrm{lb}} M * (s, t, m) \in M$$
$$* \mathsf{loop}(H[\mathbf{inr}\ (s, t, m)], e))$$

It also uses the evidence accumulation technique described in §4.3, but additionally accumulates the evidence that a node can delay message receipt without changing behavior. The idea is that if a message m is received at some time T, then if we delay that receipt to some later time T', it is still possible to receive m. This is because the set of messages is monotonically growing. The evidence that a later recv can return a given message is accumulated in the *least fixed point* loop. Intuitively, $\mathsf{loop}(e_0, e)$ allows e_0 to execute to e in three ways: (1) Some pure steps. (2) First raising a recv effect that receives nothing and then continuing with the result of recv. (3) First raising a recv effect that receives some message (s, t, m) and then continuing with the result of recv. Assertion $t \rightsquigarrow^{\mathrm{lb}} M$ in cases (2) and (3) is a lower-bound resource of $t \rightsquigarrow \cdot$, meaning that M is a subset of messages that have ever been sent to address t.

E Eliminating Primitive Effects via Pre-Determinism

Although FicusLang does not come with built-in mutable state, it is not strictly pure because it includes two primitive effects: **pick** is not deterministic and **observe** adds a special label to the transition relation. However, it turns out that even these primitives can be removed; we kept them in the main body of the paper for simplicity of presentation. In this section, we present an alternative core calculus PureFicusLang that is strictly pure and a logic for it called PureFicus. PureFicusLang is identical to FicusLang except that the former does not have **pick** and **observe** primitives. Instead, it adds a special **label** primitive. The **label** primitive is a trivial, pure operation. It takes an integer

as an argument and simply reduces to unit: $\textbf{label}\ z \to_{\mathsf{h}} ()$. Its role is simply as a syntactic marker for annotating certain points of the program.

Then, within PureFicusLang, we model **pick** and **observe** also as effect handlers. The non-determinism of **pick** is simulated by using a handler parameterized by a list of booleans that will be returned to invocations of **pick**. Once the language is made deterministic, the prophecy behavior needed for **observe** becomes trivial: there is nothing to "predict" with prophecies, since the program's future behavior is entirely predetermined. Below, we first explain in more detail the protocol and implementation of these handlers. Finally, we justify the soundness of this encoding by connecting it back to the original FicusLang operational semantics in which these effects were primitives, thereby showing that it is equivalent.

E.1 Non-Determinism

We first define the protocol $\mathsf{PICK}(v, \Phi) \triangleq v = (\mathtt{pick}, ()) * (\forall b \in \mathbb{B}.\ \Phi(b))$, and set $\mathtt{pick} \triangleq \textbf{do}\ (\mathtt{pick}, ())$. It is straightforward to verify that the new pick in PureFicusLang satisfies the same client-side reasoning rule as the primitive **pick** in FicusLang. The handler $\mathtt{run}_{\mathtt{pick}}$ takes as parameter a list of booleans for the future choices to return. Each time a pick effect is raised, the handler pops the value from the head of the list and returns it. In case the list is used up, the handler simply diverges, leading to a trivially safe program under partial correctness. The implementation of $\mathtt{run}_{\mathtt{pick}}$ is shown in §G.13. The specification Ewp-PickRun then guarantees that if a program *main* is verified under the PICK protocol, then an ewp holds for the handler when using any list of booleans.

$$\textsc{Ewp-PickRun}\quad \frac{\mathtt{pick} \notin \mathsf{tags}(\Psi) \qquad \mathsf{ewp}_{\mathrm{W}}\ \mathit{main}\ ()\ \langle \Psi \oplus \mathsf{PICK} \rangle\ \{\Phi\}}{\forall\, \vec{b} \in \mathbb{B}^*.\ \mathsf{ewp}_{\mathrm{W}}\ \mathtt{run}_{\mathtt{pick}}\ \mathit{main}\ \vec{b}\ \langle \Psi \rangle\ \{\Phi\}}\ *$$

Unlike the standard non-determinism semantics where the choice of the value being picked is delayed to the time the **pick** operation is executed, this handler essentially chooses all non-deterministic values upfront, even before any operation is executed. Nevertheless, for verifying safety properties, our model is sound *w.r.t.* the primitive model of non-determinism used in the main body, because given any prefix of an execution under the primitive non-determinism model, there exists a list of booleans that would yield the same behavior when using the handler. Thus, because $\vec{b}$ is universally quantified over all such lists, the conclusion of Ewp-PickRun covers all possible prefixes of executions of a program, which is all that is needed for partial correctness and safety properties.

E.2 Global Prophecy

Next, to encode the global prophecy rules, we define $\mathtt{observe}\ w \triangleq \textbf{do}(\mathtt{observe}, w)$ and

$$\begin{aligned}\mathsf{OBSERVE}(v, \Phi) \triangleq \exists w, \vec{v}\,.\ v = (\mathtt{observe}, w) * \mathsf{primProph}(\vec{v}) \\ * (\forall \vec{v}'.\ \vec{v} = w :: \vec{v}' * \mathsf{primProph}(\vec{v}\,') \mathrel{-\!\!*} \Phi(()))\end{aligned}$$

Here, $\mathsf{primProph}(\vec{v})$ is just a regular piece of authoritative ghost state defined as $\boxed{\circ\, \vec{v}}^{\gamma_p}$, where γ_p is some globally fixed ghost name allocated at the time the handler is installed.

Meanwhile, the handler for this protocol is effectively just a no-op: to handle an effect of $(\mathtt{observe}, v)$, it evaluates $(\textbf{label}\ \mathtt{observe}, v)$ to syntactically "mark" the observation effect, discards the result, and calls the continuation with $()$. As alluded to above, at the logic level, because the language is pure and thus deterministic, the future execution behavior of the program, and thus

all of the reductions of **label** that will occur, are pre-determined. Therefore, we can simply "pre-compute" upfront what this trace of reductions will be at the time the handler is installed.[9] Because that pre-computed trace is the only possible trace the program can yield, when an observation effect is raised, the value of that observation must match the value predicted by the global prophecy.

To carry out this argument, we extend the program logic with additional ghost state. The core new feature is a ghost state mechanism we call *snapshot*, which comes with a $\mathsf{snapshot}(e)$ resource witnessing that the program was e at some point in the past. Additionally, we also need permissions on labels and time receipts, and a way to track the surrounding evaluation context that an expression is evaluating under in ewp. The details about these features are explained in §F.1 and a proof based on these features is outlined in §F.3.

E.3 Soundness

Similar to Theorem 5.1, there is also an adequacy theorem for PureFicus.

Theorem E.1 (Adequacy, PureFicus). *Let φ be a first-order predicate. If* $\mathsf{labelEn}(\top) \vdash \mathsf{ewp}\ e\ \langle\bot\rangle\ \{\varphi\}$, *then executing e never gets stuck, and if $e \to^* v$ then $\varphi(v)$ holds.*

Here, $\mathsf{labelEn}(\top)$ is a token saying that all **label** expressions are permitted to execute.

Further, we reconstruct FicusLang using PureFicusLang, and prove that for every execution in FicusLang, there exists a corresponding execution in PureFicusLang that produces the same result. To state this result formally, in FicusLang, define trivial effect handlers $\mathsf{run}'_{\mathsf{pick}}$ and $\mathsf{run}'_{\mathsf{observe}}$ that simply perform the primitive **pick** and **observe**. These similarly implement protocols PICK and OBSERVE respectively using the primitive rules for these effects.

Let H_p be a context in PureFicusLang that handles the non-determinism and global prophecy effects and H_f be a context in FicusLang with $\mathsf{run}'_{\mathsf{pick}}$ and $\mathsf{run}'_{\mathsf{observe}}$ installed.

$$H_p(\vec{b}) \triangleq \mathsf{run}_{\mathsf{observe}}\ (\lambda_.\ \mathsf{run}_{\mathsf{pick}}\ (\lambda_.\ \bullet)\ \vec{b})$$
$$H_f \triangleq \mathsf{run}'_{\mathsf{observe}}\ (\lambda_.\ \mathsf{run}'_{\mathsf{pick}}\ (\lambda_.\ \bullet))$$

Then, we have the following theorem.

Theorem E.2 (Model Soundness). *Let e be an expression of FicusLang that does not contain* **pick** *and* **observe** *language primitives, and $\ll e \gg$ be the equivalent expression of PureFicusLang. If evaluating $H_p(\vec{b})[\ll e \gg]$ never gets stuck for all $\vec{b}$, then,*

- *evaluating $H_f[e]$ never gets stuck;*
- *if $H_f[e] \to^* v$, then there exists a boolean list $\vec{b}$ such that $H_p(\vec{b})[\ll e \gg] \to^* \ll v \gg$.*

This shows that any safety property verified under PureFicusLang also holds under FicusLang.

F PureFicus

F.1 Reasoning Rules

LabelEn-DisjUnion
$$\mathsf{labelEn}(L_1 \uplus L_2) \dashv\vdash \mathsf{labelEn}(L_1) * \mathsf{labelEn}(L_2)$$

Ewp-Label
$$\frac{\mathsf{labelEn}(L) \qquad z \in L}{\mathsf{ewp}_{\mathrm{W}}\ \mathbf{label}\ z\ \langle\Psi\rangle\ \{_.\ \mathsf{labelEn}(L)\}}\ *$$

EwpEctx-Bind
$$\frac{\mathsf{ewp}^{K_o \mathbin{+\!\!+} N}_{\mathrm{W}_1,\mathrm{W}_2}\ e\ \langle\Psi\rangle\ \{v.\ \mathsf{ewp}_{\mathrm{W}_2,\mathrm{W}_3}\ N[v]\ \langle\Psi\rangle\ \{\Phi\}\}}{\mathsf{ewp}^{K_o}_{\mathrm{W}_1,\mathrm{W}_3}\ N[e]\ \langle\Psi\rangle\ \{\Phi\}}\ *$$

EwpEctx-PureBind
$$\frac{\mathsf{ewp}^{K_o \mathbin{+\!\!+} K}_{\mathrm{W}_1,\mathrm{W}_2}\ e\ \langle\bot\rangle\ \{v.\ \mathsf{ewp}^{K_o}_{\mathrm{W}_2,\mathrm{W}_3}\ K[v]\ \langle\Psi\rangle\ \{\Phi\}\}}{\mathsf{ewp}^{K_o}_{\mathrm{W}_1,\mathrm{W}_3}\ K[e]\ \langle\Psi\rangle\ \{\Phi\}}\ *$$

[9] In our Rocq formalization, this step uses the law of excluded middle.

$$\textsc{Ewp-EctxFreeze}\quad \frac{\forall K.\ \mathsf{ewp}^{K}_{W_1,W_2}\ e\ \langle\Psi\rangle\ \{\Phi\}}{\mathsf{ewp}_{W_1,W_2}\ e\ \langle\Psi\rangle\ \{\Phi\}}\ast \qquad \textsc{Ewp-EctxUnfreeze}\quad \frac{\mathsf{ewp}_{W_1,W_2}\ e\ \langle\Psi\rangle\ \{\Phi\}}{\mathsf{ewp}^{K}_{W_1,W_2}\ e\ \langle\Psi\rangle\ \{\Phi\}}\ast$$

$$\textsc{Ewp-SnapshotCreate}\quad \frac{e \notin \mathit{Val} \cup \mathit{Eff} \qquad \mathsf{labelEn}(\{l\}) \qquad \mathsf{snapshot}^{l}(K[e]) \mathrel{-\!\!*} \mathsf{ewp}^{K}_{W_1,W_2}\ e\ \langle\Psi\rangle\ \{\Phi\}}{\mathsf{ewp}^{K}_{W_1,W_2}\ e\ \langle\Psi\rangle\ \{\Phi\}}\ast$$

$$\textsc{Ewp-SnapshotSteps}\quad \frac{e \notin \mathit{Val} \cup \mathit{Eff} \qquad \mathsf{snapshot}^{l}(e_0) \qquad e_0 \xrightarrow[\top\setminus\{l\}]{}{}^{*} K[e] \ast \mathsf{snapshot}^{l}(e_0) \mathrel{-\!\!*} \mathsf{ewp}^{K}_{W_1,W_2}\ e\ \langle\Psi\rangle\ \{\Phi\}}{\mathsf{ewp}^{K}_{W_1,W_2}\ e\ \langle\Psi\rangle\ \{\Phi\}}\ast$$

$$\textsc{Ewp-SnapshotDestroy}\quad \frac{e \notin \mathit{Val} \cup \mathit{Eff} \qquad \mathsf{snapshot}^{l}(e_0) \qquad \mathsf{labelEn}(\{l\}) \mathrel{-\!\!*} \mathsf{ewp}^{K}_{W_1,W_2}\ e\ \langle\Psi\rangle\ \{\Phi\}}{\mathsf{ewp}^{K}_{W_1,W_2}\ e\ \langle\Psi\rangle\ \{\Phi\}}\ast \qquad \textsc{TR-Zero}\quad \vdash \Rrightarrow \mathsf{TR}(0)$$

$$\textsc{TR-Plus}\quad \mathsf{TR}(m+n) \dashv\vdash \mathsf{TR}(m) \ast \mathsf{TR}(n) \qquad \textsc{Ewp-MaxSteps}\quad \frac{e \notin \mathit{Val} \cup \mathit{Eff} \qquad \forall m.\ \mathsf{maxsteps}(m) \mathrel{-\!\!*} \mathsf{ewp}_{W_1,W_2}\ e\ \langle\Psi\rangle\ \{\Phi\}}{\mathsf{ewp}_{W_1,W_2}\ e\ \langle\Psi\rangle\ \{\Phi\}}\ast$$

$$\textsc{Ewp-Timeout}\quad \frac{e \notin \mathit{Val} \cup \mathit{Eff} \qquad \mathsf{maxsteps}(n) \qquad \mathsf{TR}(n)}{\mathsf{ewp}_{W_1,W_2}\ e\ \langle\Psi\rangle\ \{\Phi\}}\ast$$

Permission on Labels. Although **`label`** is just a nop semantically, PureFicus treats it specially: the logic comes with a token $\mathsf{labelEn}(L)$ controlling which labels are permitted to execute, where $L \in \wp(\mathbb{Z})$. This token can be split into smaller permission tokens using LabelEn-DisjUnion. To prove an ewp of **`label`** z, one must supply a $\mathsf{labelEn}(L)$ token such that $z \in L$.

***Context-Aware* ewp.** When reasoning about a large program, we often use Ewp-Bind to focus on a fragment of this program under a (neutral) evaluation context. Essentially, this rule reduces a global ewp about the whole program $N[e]$ into a local ewp about a subprogram e, and when verifying e, we lose track of the surrounding evaluation context N. PureFicus strengthens ewp by adding another parameter K to keep track of the outside evaluation context of the current focused expression: $\mathsf{ewp}^{K}_{W_1,W_2}\ e\ \langle\Psi\rangle\ \{\Phi\}$. This strengthened ewp is called context-aware ewp. Most reasoning rules for ewp still hold for the context-aware ewp, except that the bind rule becomes EwpEctx-Bind and EwpEctx-PureBind.

EwpEctx-Bind is the context-aware analogue of Ewp-Bind: because we zoomed into a subprogram, the evaluation context changes from K_o to $K_o +\!\!+ N$. Critically, after finishing verifying e, we have to forget the outside evaluation context and continue with a regular context-unaware ewp of $N[v]$. This is because if e raises an effect, the handler can change the outside evaluation context. If e never raises an effect, we have a stronger EwpEctx-PureBind rule that preserves the outside evaluation context after e finishes. The regular ewp is equivalent to a context-aware ewp under an arbitrary context.

Snapshots. A $\mathsf{snapshot}^{l}(e)$ is a resource to memorize the execution history of the program. In its reasoning rules, $e_1 \underset{L}{\rightarrow} e_2 \triangleq e_1 \rightarrow e_2 \wedge (\text{if } \mathit{first\text{-}redex}(e_1) \text{ is } \textbf{`label`}\ z \text{ then } z \in L)$, and $\underset{L}{\rightarrow}^{*}$ is

the reflexive transitive closure of $\xrightarrow[L]{}$. To create a snapshot, one must supply a $\mathsf{labelEn}(\{l\})$ token. Then, the snapshot resource $\mathsf{snapshot}^l(e_0)$ guarantees that the current expression is reachable from e_0, and **label** l was never executed since $\mathsf{snapshot}^l(e_0)$ was created. To regain the permission to execute **label** l, one must destroy the snapshot.

Time Receipts. Because we work on partial correctness, it is sufficient to consider an arbitrarily long (but finite) execution of a program. While step-indexing and Löb induction already *internalize* this reasoning principle into the Iris logic, PureFicus further *externalizes* it using time receipts [41].

When reasoning in PureFicus, we can assume there is a resource $\mathsf{maxsteps}(m)$. Each step of execution produces a time receipt $\mathsf{TR}(1)$, and with m time receipts, we immediately prove any ewp.

F.2 Model

$$\mathsf{ewp}_{W_1,W_2}\ e\ \langle\Psi\rangle\ \{\Phi\} \triangleq \forall K.\ \mathsf{ewp}^K_{W_1,W_2}\ e\ \langle\Psi\rangle\ \{\Phi\}$$

$$\mathsf{ewp}^K_{W_1,W_2}\ v\ \langle\Psi\rangle\ \{\Phi\} \triangleq {}_{W_1}\Rrightarrow_{W_2} \Phi(v)$$

$$\mathsf{ewp}^K_{W_1,W_2}\ \S(N)[v]\ \langle\Psi\rangle\ \{\Phi\} \triangleq {}_{W_1}\Rrightarrow_{\bot} \Psi(v, \lambda w.\ \Rrightarrow_{\bot} \triangleright(\mathsf{ewp}_{\bot,W_2}\ N[w]\ \langle\Psi\rangle\ \{\Phi\}))$$

$$\mathsf{ewp}^K_{W_1,W_2}\ e\ \langle\Psi\rangle\ \{\Phi\} \triangleq \forall n, m, L.\ \mathsf{TR}_\bullet(n) * \mathsf{maxsteps}(m) * \mathsf{snapshot}^L_\bullet(K[e]) \mathrel{-\!\!*} {}_{W_1}\Rrightarrow_{\bot}$$
$$n \geq m \vee \mathsf{red}(e) * \forall e'.\ e \longrightarrow e' \mathrel{-\!\!*} \pounds(1) \mathrel{-\!\!*} \Rrightarrow_{\bot} \triangleright \Rrightarrow_{\bot}$$
$$\mathsf{TR}_\bullet(1+n) * (\exists L'.\ \mathsf{snapshot}^{L'}_\bullet(K[e'])) * \mathsf{ewp}^K_{\bot,W_2}\ e'\ \langle\Psi\rangle\ \{\Phi\}$$

$$\mathsf{TR}_\bullet(n) \triangleq \boxed{\bullet\mathsf{nat}(n)}^{\gamma_{tr}} \qquad \mathsf{TR}(n) \triangleq \boxed{\circ\mathsf{nat}(n)}^{\gamma_{tr}} \qquad \mathsf{maxsteps}(m) \triangleq \boxed{\mathsf{ag}(m)}^{\gamma_{ms}}$$

$$\mathsf{snapshot}^L_\bullet(e) \triangleq \exists M.\ \boxed{\bullet\mathsf{ex}\ \langle\$\rangle\ M}^{\gamma_{snapshot}} * \boxed{\bullet\mathsf{set}(L)}^{\gamma_{labelEn}} * \mathop{\Large *}_{l \mapsto e_0 \in M} e_0 \xrightarrow[\top\setminus\{l\}]{}{}^* e * l \notin L$$

$$\mathsf{snapshot}^l(e) \triangleq \boxed{\circ\,[l \leftarrow \mathsf{ex}(e)]}^{\gamma_{snapshot}} \qquad \mathsf{labelEn}(L) \triangleq \boxed{\circ\mathsf{set}(L)}^{\gamma_{labelEn}}$$

F.3 Verifying the Effect Handler $\mathsf{run}_{\mathtt{observe}}$

Define

$$I \triangleq \exists \vec{v}, e, m, n.\ \mathsf{primProph}_\bullet(\vec{v}) * \mathsf{snapshot}^{\mathtt{observe}}(e) * \mathsf{TR}(n) * \mathsf{maxsteps}(m) * n \leq m * \mathsf{futureobs}(e, m-n, \vec{v})$$

$$\mathsf{futureobs}(_, 0, \vec{v}) \triangleq \vec{v} = \epsilon$$

$$\mathsf{futureobs}(e, 1+n, \vec{v}) \triangleq \Big(\exists K, w, \vec{v}\,'.\ e \xrightarrow[\top\setminus\{\mathtt{observe}\}]{}{}^* K[(\mathbf{label}\ \mathtt{observe}, w)] \wedge \vec{v} = w :: \vec{v}\,' \wedge \mathsf{futureobs}(K[((), w)], n, \vec{v}\,')\Big)$$
$$\vee \neg\Big(\exists K, w.\ e \xrightarrow[\top\setminus\{\mathtt{observe}\}]{}{}^* K[(\mathbf{label}\ \mathtt{observe}, w)]\Big) \wedge \vec{v} = \epsilon$$

We are now able to prove this specification by Löb induction and maintaining I as an invariant.

Ewp-ObserveRun

$$\frac{\mathtt{observe} \notin \mathsf{tags}(\Psi) \qquad \mathtt{observe} \in L \qquad \mathsf{labelEn}(L) \qquad \forall \vec{v}.\ \mathsf{labelEn}(L \setminus \{\mathtt{observe}\}) \mathrel{-\!\!*} \mathsf{primProph}(\vec{v}) \mathrel{-\!\!*} \mathsf{ewp}_W\ main\ ()\ \langle\Psi \oplus \mathrm{OBSERVE}\rangle\ \{\Phi\}}{\mathsf{ewp}_W\ \mathsf{run}_{\mathtt{observe}}\ main\ \langle\Psi\rangle\ \{\Phi\}}\ *$$

Notably, we use the law of excluded middle to argue that given an expression e and natural number n, there exists a $\vec{v}$ such that $\mathsf{futureobs}(e, n, \vec{v})$.

G Implementation of Handlers

G.1 State

```
run_state ≜ λmain init. go main () init
where go ≜ rec go k r σ.                    (* r is the result of last operation; σ is the global state *)
1            try k r with
2              v k    ⇒ (match v with
3                           (read, ()) ⇒ go k σ σ                (* returns the current state *)
4                         | (write, y) ⇒ go k () y               (* updates the state to y *)
5                         |      (et, v) ⇒ go k (do (et, v)) σ   (* re-raises the effect *)
6            | ret v ⇒ v
```

G.2 Heap

```
run_heap ≜ λmain.
1  write (0, ∅);
2  deep-try main () with ret v ⇒ v | v k ⇒ match v with
3        (alloc, v) ⇒ let (ℓ, H) := read in write (ℓ + 1, H[ℓ ← v]); k ℓ
4  |      (load, ℓ) ⇒ let (_, H) := read in k H(ℓ)
5  | (store, (ℓ, w)) ⇒ let (n, H) := read in write (n, H[ℓ ← w]); k ()
6  |      (free, ℓ) ⇒ let (n, H) := read in write (n, H \ {ℓ}); k ()
7  |         (et, v) ⇒ k (do (et, v))
```

G.3 Atomic Heap

```
run_atomheap ≜ λmain.
1  deep-try main () with ret v ⇒ v | v k ⇒ match v with
2    (cas, (ℓ, v, w)) ⇒ let v₀ :=!ℓ in
                         if v₀ = v then ℓ ← w; k(v₀, true) else k (v₀, false)
3  |  (xchg, (ℓ, v)) ⇒ let v₀ :=!ℓ in ℓ ← v; k v₀
4  |   (faa, (ℓ, j)) ⇒ let i :=!ℓ in ℓ ← i + j; k i
5  |         (et, v) ⇒ k (do (et, v))
```

G.4 Concurrency

```
run_conc ≜ λmain. go {|(main, (), M)|}                (* create a new singleton bag *)
where go ≜ rec go pool.                               (* pool is the thread pool *)
1            let ((k, r, t), pool) := choose pool in  (* choose one thread from pool *)
2            try k r with
3              v k    ⇒ (match v with
4                           (fork, e) ⇒ go ({|(e, (), C), (k, (), t)|} ⊎ pool)
5                         |    (et, v) ⇒ go ({|(k, do (et, v), t)|} ⊎ pool))
6            | ret v ⇒ if t = M^× then v
7                      else go ({|((λ_. v), (), t^×)|} ⊎ pool)
```

G.5 Prophecy

```
run_proph ≜ λmain.
1  write 0;
2  deep-try main () with ret v ⇒ v | v k ⇒ match v with
3                (newproph, ()) ⇒ let p := read in write (p + 1); k p
4  | (resolve_proph, (v, p, w)) ⇒ observe (p, v, w); k v
5  |                    (et, v) ⇒ k (do (et, v))
```

G.6 Atomic Prophecy

```
run_atomproph ≜ λmain.
deep-try main () with ret v ⇒ v | v k ⇒ match v with
  (resolve, (v, p, w)) ⇒ k (do (resolve_proph, (do v, p, w)))
| (et, v) ⇒ k (do (et, v))
```

G.7 Network

```
run_network ≜ λmain.
write [a ← ∅ | a ∈ a⃗] ;
deep-try main () with ret v ⇒ v | v k ⇒ match v with
  (send, (s, t, m)) ⇒ let net := read in
    write (net[t ←({(s, m)} ∪ net(t))]); k ()
| (recv, t) ⇒ let net := read in
    (if net(t) = ∅ || nondet_bool () then k (inl ())
    else let ((s, m), _) := choose net(t) in k (inr (s, t, m)))
| (et, v) ⇒ k (do (et, v))
```

G.8 Distributed System

```
run_distr ≜ λmain. go {|(main, (), M)|}
where go ≜ rec go pool.
1   let ((k, r, t), pool) := choose pool in
2   let rec loop k r := (try k r with
3     v k    ⇒ (match v, t with
4                 (start, e), M ⇒ go ({|(e, (), C), (k, (), M)|} ⊎ pool)
5               | (start, _), _  ⇒ go ({|(k, (), t)|} ⊎ pool) (* nop *)
6               | (recv, v), _   ⇒ loop k (do (recv, v))
7               |     (et, v), _ ⇒ go ({|(k, do (et, v), t)|} ⊎ pool))
8    | ret v ⇒ if t = M^× then v
9              else go ({|((λ_. v), (), t^×)|} ⊎ pool)) in
10  loop k r
```

G.9 Crash

```
run_crash_recover ≜ rec run main.
write 0;
deep-try main () with ret v ⇒ v | v k ⇒ match v with
                  (crash, ()) ⇒ observe (); run(main)
|           (newproph, ()) ⇒ let p := read in write (p + 1); k p
| (resolve_proph, (v, p, w)) ⇒ observe (p, v, w); k v
|                    (et, v) ⇒ k (do (et, v))
```

G.10 Crash Trigger

```
run_crash_trigger ≜ λmain.
if nondet_bool () then do (crash, ())
else deep-try main () with
    v k ⇒ let r := do v in
          if nondet_bool () then do (crash, ())
          else k r
| ret v ⇒ v
```

G.11 Disk

```
run_disk ≜ λmain.
1  write ∅;
2  deep-try main () with ret v ⇒ v | v k ⇒ match v with
3          (disk_load, ℓ) ⇒ let D := read in k D(ℓ)
4  | (disk_store, (ℓ, w)) ⇒ let D := read in write (D[ℓ ← w]); k ()
5  |                (et, v) ⇒ k (do (et, v))
```

G.12 Asynchronous Disk

```
run_adisk ≜ λmain.
1  write ∅;
2  deep-try main () with ret v ⇒ v | v k ⇒ match v with
3    (adisk_load, ℓ) ⇒ let buf := read in
4        let r := (if ℓ ∈ buf then fst buf(ℓ) else disk_load ℓ) in
5        partial_flush (); k r
6  | (adisk_store, (ℓ, w)) ⇒ let buf := read in
7        (if ℓ ∈ buf then resolve_proph (snd buf(ℓ)) to false);
8        let p := newproph in write(buf[ℓ ←(w, p)]); partial_flush (); k ()
9  | (adisk_barrier, ()) ⇒ let buf := read in
10       iter write_back buf; write ∅; k ()
11 | (et, v) ⇒ k (do (et, v))
write_back ≜ λℓ, (v, p). resolve_proph p to true; disk_store ℓ v
partial_flush ≜ λ(). let buf := read in
       write (filter (λℓ, (v, p). if pick then (write_back ℓ (v, p); false) else true) buf)
```

G.13 Non-Determinism

```
run_pick ≜ λmain entropy. go main () entropy
where go ≜ rec go k r b⃗ .
1              try k r with
2                 v k  ⇒ (match v, b⃗ with
3                              (pick, ()), b0 :: b⃗′ ⇒ go k b0 b⃗′
4                            |      (pick, ()), ϵ ⇒ diverge ()
5                            |          (et, v), b⃗ ⇒ go k (do (et, v)) b⃗)
6               | ret v ⇒ v
```

G.14 Global Prophecy

```
run_observe ≜ λmain.
1  deep-try main () with ret v ⇒ v | v k ⇒ match v with
2    (observe, v) ⇒ (label observe, v); k ()
3  |       (et, v) ⇒ k (do (et, v))
```